

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN I

BỘ MÔN IOT VÀ ỨNG DỤNG



BÁO CÁO BÀI TẬP IOT VÀ ỨNG DỤNG

ĐỀ TÀI: XÂY DỰNG HỆ THỐNG CẢM BIẾN IOT

Giảng viên hướng dẫn	: Nguyễn Quốc Uy
Họ và tên sinh viên	: Nguyễn Duy Thanh
Mã sinh viên	: B23DCCN761
Lớp	: 11

Hà Nội – 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU.....	5
I. Mục đích của đề tài.....	5
II. Các chức năng chính.....	5
CHƯƠNG 2: THIẾT KẾ TỔNG QUAN.....	6
I. Kiến trúc tổng quan của hệ thống.....	6
II. Công nghệ sử dụng.....	6
III. Phần cứng sử dụng.....	7
CHƯƠNG 3: THIẾT KẾ CHI TIẾT.....	8
I. Cơ sở dữ liệu.....	8
1. Bảng users.....	8
2. Bảng sensors.....	9
3. Bảng datasensors.....	10
4. Bảng devices.....	10
5. Bảng actions.....	11
II. Giao diện.....	12
III. Use case.....	14
A. Use case 1: Theo dõi dữ liệu cảm biến.....	14
1. Giao diện.....	14
2. Actor.....	15
3. Mô tả use case.....	15
4. Biểu đồ tuần tự.....	15
5. API documents.....	18
B. Use case 2: Bật tắt đèn LED.....	20
1. Giao diện.....	20
2. Actor.....	20
3. Mô tả usecase.....	20
4. Biểu đồ tuần tự.....	21
5. API documents.....	23
C. Use case 3: Tra cứu lịch sử.....	26
1. Giao diện.....	26
2. Actor.....	27
3. Mô tả usecase.....	27
4. Biểu đồ tuần tự.....	27
5. API document.....	28
CHƯƠNG 4: THIẾT KẾ VÀ CÀI ĐẶT PHẦN CỨNG.....	32
I. Thiết kế phần cứng.....	32

1. Bảng quy hoạch chân kết nối.....	32
2. Diễn giải nguyên lý đấu nối.....	33
II. Mã Nguồn Điều Khiển Thiết Bị (ESP8266).....	34
III. Luồng điều khiển thiết bị.....	42
1. Nạp mã nguồn từ Arduino vào ESP8266.....	42
2. Khởi động máy chủ MQTT.....	42
3. Luồng nhận dữ liệu cảm biến.....	43
4. Luồng điều khiển đèn LED và nhận kết quả trạng thái.....	43
CHƯƠNG 5: MÃ NGUỒN.....	45
I. Back-end.....	45
1. Công nghệ và kiến trúc hệ thống.....	45
2. Cấu trúc thư mục back-end.....	47
3. Tầng dữ liệu và ánh xạ thực thể (Entity Mapping).....	47
4. Danh mục API và giao thức truyền thông.....	48
5. Cấu hình và xử lý lỗi tập trung.....	48
II. Front-end.....	49
1. Công nghệ và kiến trúc phía Client.....	49
2. Cấu trúc thư mục Front-end.....	49
3. Điều hướng Ứng dụng (Routing).....	50
4. Tầng giao tiếp dữ liệu: REST API Client & WebSocket Service.....	50
5. Quản lý trạng thái (State Management) & đồng bộ bộ nhớ.....	50
III. Các luồng nghiệp vụ cốt lõi.....	51
1. Kiến trúc tương tác tổng quát.....	51
2. Luồng 1: Thu thập và giám sát cảm biến thời gian thực.....	51
3. Luồng 2: Điều khiển thiết bị kết hợp hẹn giờ timeout 15s và cơ chế Rollback.....	52
4. Luồng 3: Tra cứu và phân trang lịch sử cảm biến.....	54
IV. Cài đặt và triển khai hệ thống.....	56
1. Yêu cầu môi trường.....	56
2. Khởi chạy phía Back-end.....	56
3. Khởi chạy phía Front-end.....	56
CHƯƠNG 6: KẾT LUẬN.....	56
I. Tổng kết các kết quả đạt được.....	56
II. Hướng phát triển.....	57

CHƯƠNG 1: GIỚI THIỆU

I. Mục đích của đề tài

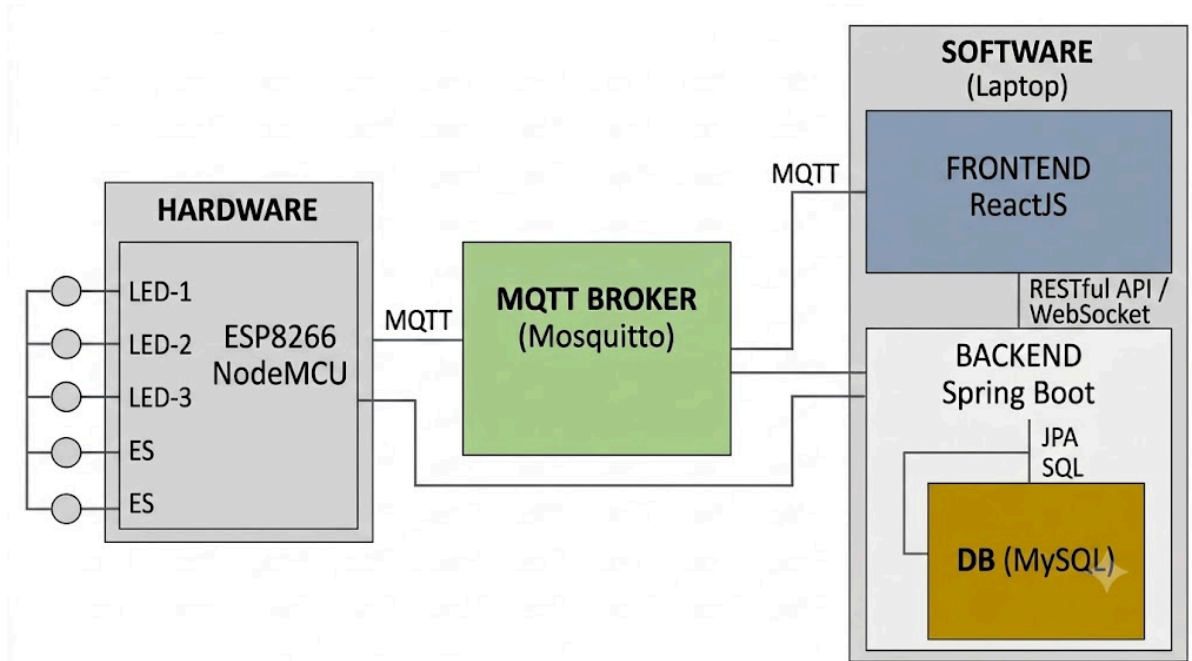
- Internet of Things (IoT) là mô hình kết nối các thiết bị vật lý với Internet nhằm thu thập, truyền tải và xử lý dữ liệu. Trong đó, các cảm biến có thể thu thập thông tin từ môi trường thực tế, đồng thời người dùng có thể giám sát và điều khiển thiết bị thông qua các ứng dụng phần mềm.
- Đề tài được thực hiện nhằm xây dựng một hệ thống IoT giám sát và điều khiển thiết bị, trong đó vi điều khiển ESP8266 kết nối với các cảm biến để thu thập các thông tin về nhiệt độ, độ ẩm và ánh sáng. Dữ liệu sau khi được thu thập sẽ được truyền về hệ thống Backend thông qua giao thức MQTT, sau đó được lưu trữ và cung cấp cho giao diện Web để người dùng theo dõi.
- Bên cạnh chức năng giám sát, hệ thống còn cho phép người dùng điều khiển bật/tắt đèn LED từ máy tính thông qua giao diện Web. Qua đó, đề tài minh họa quá trình kết hợp giữa phần cứng IoT, giao thức truyền thông, cơ sở dữ liệu và ứng dụng Web trong một hệ thống hoàn chỉnh.

II. Các chức năng chính

- Hệ thống được xây dựng với các chức năng chính sau:
 - + Cảm biến thời gian thực: hiển thị các giá trị nhiệt độ, độ ẩm và ánh sáng được cập nhật từ ESP8266.
 - + Biểu đồ: biểu diễn sự biến thiên của nhiệt độ, độ ẩm và ánh sáng theo thời gian, giúp người dùng dễ dàng theo dõi xu hướng thay đổi của môi trường.
 - + Bật/tắt thiết bị: cho phép người dùng điều khiển trạng thái của đèn LED thông qua giao diện Web.
 - + Lịch sử cảm biến: hiển thị các dữ liệu cảm biến đã được thu thập, hỗ trợ phân trang, lọc theo loại cảm biến và khoảng thời gian.
 - + Lịch sử bật/tắt: hiển thị lịch sử điều khiển thiết bị, hỗ trợ phân trang, lọc theo thiết bị, trạng thái và khoảng thời gian.

CHƯƠNG 2: THIẾT KẾ TỔNG QUAN

I. Kiến trúc tổng quan của hệ thống



Hình 1: Kiến trúc tổng quan của hệ thống

II. Công nghệ sử dụng

1. Frontend: ReactJs

- ReactJS được sử dụng để xây dựng giao diện Web của hệ thống. Frontend chịu trách nhiệm hiển thị dữ liệu cảm biến, biểu đồ, giao diện điều khiển thiết bị và các bảng lịch sử.
- Frontend giao tiếp với Backend thông qua:
 - + REST API đối với các chức năng truy vấn và điều khiển.
 - + WebSocket đối với các chức năng cần cập nhật dữ liệu theo thời gian thực.

2. Backend: Spring Boot

- Spring Boot được sử dụng để xây dựng Backend và cung cấp các API theo kiến trúc RESTful API.

3. MQTT: Mosquitto

- Trong hệ thống, Mosquitto được sử dụng làm MQTT Broker để trung gian truyền nhận các message giữa ESP8266 và Backend.
- MQTT được sử dụng cho ba mục đích chính:
 - + ESP8266 publish dữ liệu cảm biến lên MQTT Broker.
 - + Backend publish các lệnh điều khiển thiết bị đến ESP8266.
 - + ESP8266 publish dữ liệu trạng thái thiết bị lên MQTT Broker.

4. WebSocket

- WebSocket được sử dụng để hỗ trợ truyền dữ liệu theo thời gian thực giữa Backend và Frontend.
- Sau khi Backend nhận được dữ liệu cảm biến mới từ MQTT, dữ liệu có thể được gửi ngay đến Frontend thông qua WebSocket mà không cần Frontend liên tục gửi request REST API.

5. *Database: MySQL*

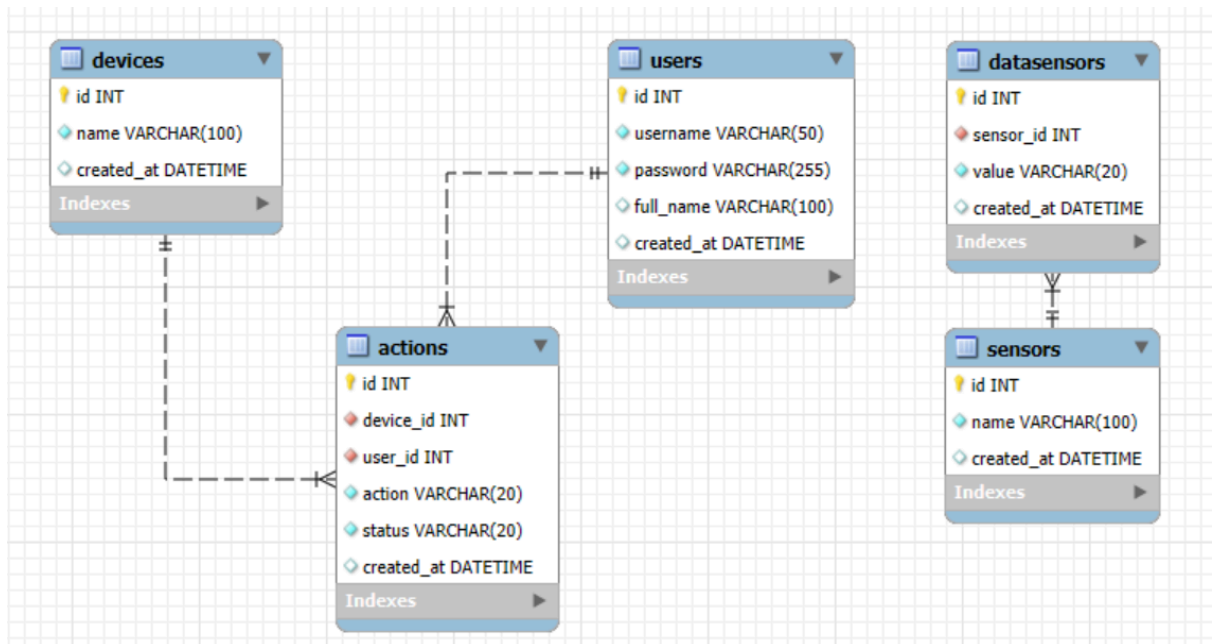
- Hệ thống sử dụng cơ sở dữ liệu quan hệ để lưu trữ thông tin người dùng, cảm biến, thiết bị, dữ liệu cảm biến và lịch sử điều khiển.
- CSDL gồm 5 bảng: users, sensors, datasensors, devices, actions.

III. **Phần cứng sử dụng**

- 1 ESP8266: Vi điều khiển trung tâm, kết nối Internet và giao tiếp với Backend thông qua MQTT.
- 1 Cảm biến nhiệt độ/độ ẩm DHT11: Thu thập dữ liệu nhiệt độ và độ ẩm môi trường.
- 1 Cảm biến ánh sáng: Thu thập dữ liệu cường độ ánh sáng.
- 3 đèn LED: Thiết bị đầu ra được sử dụng để thực hiện chức năng bật/tắt.
- Dây jumper: Kết nối các module cảm biến, LED với ESP8266
- Cáp micro USB: Dùng để kết nối laptop với ESP8266 để nạp code.
- Breadboard: Dùng để cắm linh kiện, bo mạch.
- Điện trở 220Ω: Hạn chế dòng điện đi qua đèn LED để tránh cháy nổ linh kiện, đồng thời giúp ổn định tín hiệu cho các chân GPIO khi cần thiết.

CHƯƠNG 3: THIẾT KẾ CHI TIẾT

I. Cơ sở dữ liệu



Hình 2: Cơ sở dữ liệu của hệ thống

- Cơ sở dữ liệu gồm 5 bảng:
 - + users: lưu thông tin người dùng.
 - + sensors: lưu thông tin các loại cảm biến.
 - + datasensors: lưu dữ liệu được thu thập từ cảm biến.
 - + devices: lưu thông tin các thiết bị có thể điều khiển.
 - + actions: lưu lịch sử các thao tác điều khiển thiết bị.

1. Bảng users

- Mục đích: Lưu trữ thông tin người dùng của hệ thống, phục vụ hiển thị thông tin cá nhân và xác định người dùng thực hiện thao tác điều khiển thông qua khóa ngoại user_id trong bảng actions.

STT	Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả / Công dụng
1	id	INT	PK, AUTO_INCREMENT	Định danh duy nhất cho mỗi người dùng, dùng làm khóa chính và khóa ngoại tham chiếu từ bảng actions.

2	username	VARCHAR(50)	UNIQUE, NOT NULL	Tên đăng nhập, dùng để xác thực tài khoản khi người dùng đăng nhập vào hệ thống.
3	password	VARCHAR(255)	NOT NULL	Mật khẩu đã được mã hóa (hash, ví dụ BCrypt) của tài khoản, dùng để xác thực đăng nhập. Độ dài 255 để chứa chuỗi hash.
4	full_name	VARCHAR(100)	NULLABLE	Họ tên đầy đủ của người dùng, hiển thị trên giao diện sau khi đăng nhập.
5	created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời điểm tài khoản được tạo, phục vụ thống kê và truy vết lịch sử.

Bảng 1: Bảng users

2. Bảng sensors

- Mục đích: Lưu thông tin định danh các loại cảm biến được sử dụng trong hệ thống IoT (hiện có 3 loại: nhiệt độ, độ ẩm, ánh sáng). Đây là bảng danh mục (master data), không lưu giá trị đo được.

STT	Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả / Công dụng
1	id	INT	PK, AUTO_INCREMENT	Định danh duy nhất cho mỗi loại cảm biến, dùng làm khóa ngoại tham chiếu từ bảng datasensors.
2	name	VARCHAR(100)	NOT NULL	Tên/loại cảm biến, ví dụ: "Nhiệt độ", "Độ ẩm", "Ánh sáng".
3	created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời điểm loại cảm biến được khai báo/thêm vào hệ thống.

Bảng 2: Bảng sensors

3. Bảng *datasensors*

- Mục đích: Lưu trữ các giá trị cảm biến được thu thập theo thời gian thực. Mỗi khi phần cứng (Hardware) gửi dữ liệu về hệ thống qua MQTT, Backend (Subscriber) sẽ ghi giá trị nhận được vào bảng này.

STT	Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả / Công dụng
1	id	INT	PK, AUTO_INCREMENT	Định danh duy nhất cho mỗi bản ghi dữ liệu cảm biến.
2	sensor_id	INT	FK → sensors.id, NOT NULL	Xác định bản ghi dữ liệu này thuộc loại cảm biến nào (nhiệt độ, độ ẩm hay ánh sáng).
3	value	VARCHAR(20)	NOT NULL	Giá trị đo được từ cảm biến (ví dụ: "28.5"). Kiểu VARCHAR để linh hoạt với nhiều định dạng giá trị khác nhau giữa các loại cảm biến.
4	created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thời điểm dữ liệu được ghi nhận vào hệ thống, dùng để truy vấn dữ liệu theo mốc thời gian (vẽ biểu đồ, lịch sử đo).

Bảng 3: Bảng *datasensors*

- Quan hệ: N *datasensors* — 1 *sensors* (nhiều bản ghi dữ liệu thuộc về một loại cảm biến cụ thể).

4. Bảng *devices*

- Mục đích: Lưu thông tin các thiết bị mà người dùng có thể điều khiển thông qua hệ thống. Trong dự án hiện tại là 3 đèn LED đánh số từ 1 đến 3.

STT	Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả / Công dụng
1	id	INT	PK, AUTO_INCREMENT	Định danh duy nhất cho mỗi thiết bị, dùng làm khóa ngoại tham chiếu từ bảng <i>actions</i> .
2	name	VARCHAR(100)	NOT NULL	Tên thiết bị, ví dụ: "LED 1", "LED 2", "LED 3".

3	created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMES TAMP	Thời điểm thiết bị được thêm vào hệ thống.
---	------------	----------	---	--

Bảng 4: Bảng devices

- Quan hệ: 1 devices — N actions (một thiết bị có thể được thực hiện nhiều thao tác điều khiển trong quá trình sử dụng).

5. **Bảng actions**

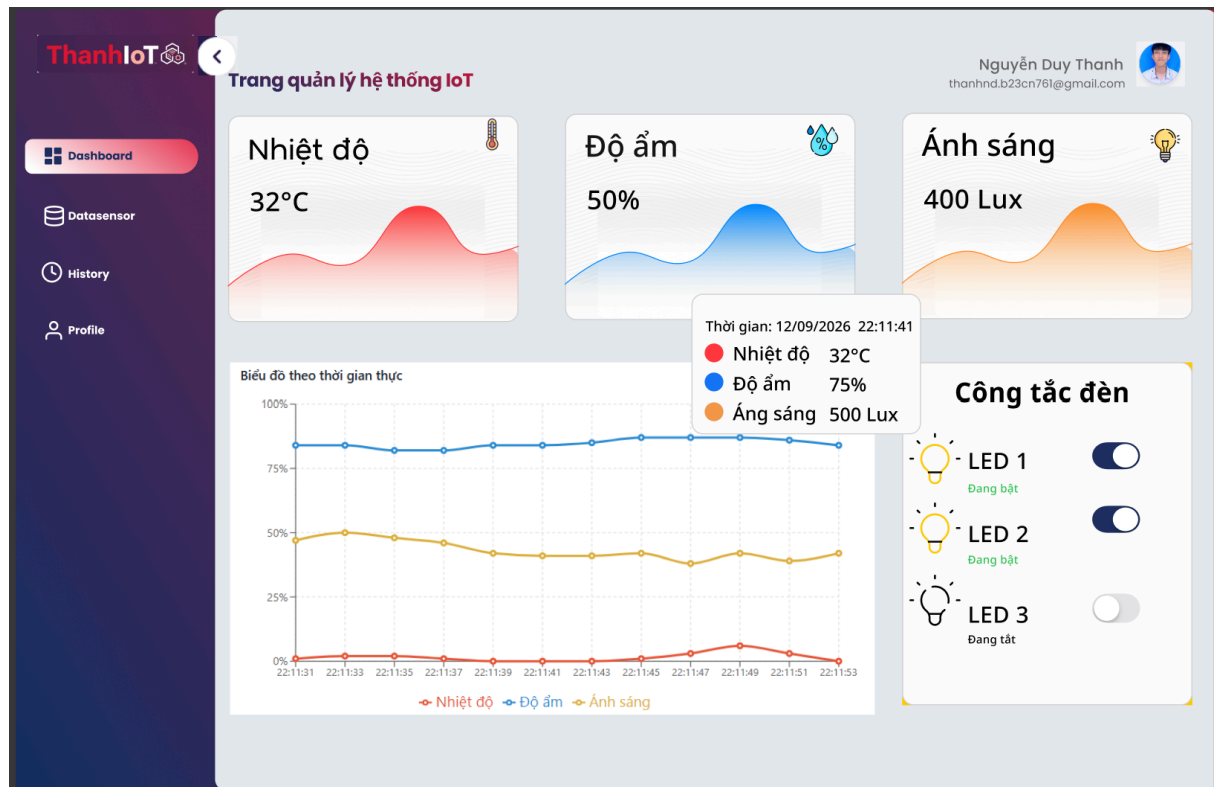
- Mục đích: Lưu lại lịch sử các thao tác điều khiển thiết bị do người dùng thực hiện trên giao diện Frontend. Khi người dùng bật/tắt thiết bị, Backend tạo một bản ghi với trạng thái ban đầu là "Pending"; sau khi lệnh được gửi qua MQTT và thiết bị phản hồi thành công, trạng thái được cập nhật thành "Success" hoặc "Failed".

STT	Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả / Công dụng
1	id	INT	PK, AUTO_INCREMENT	Định danh duy nhất cho mỗi thao tác điều khiển.
2	device_id	INT	FK → devices.id, NOT NULL	Xác định thao tác này tác động lên thiết bị nào.
3	user_id	INT	FK → users.id, NOT NULL	Xác định người dùng nào đã thực hiện thao tác điều khiển này, phục vụ truy vết/kiểm toán (audit log).
4	action	VARCHAR(20)	NOT NULL	Loại thao tác được thực hiện, ví dụ: "ON", "OFF".
5	status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'	Trạng thái xử lý của thao tác gồm Pending, Success và Failed.
6	created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMES TAMP	Thời điểm thao tác được tạo, dùng để hiển thị lịch sử điều khiển theo thời gian.

Bảng 5: Bảng actions

- Quan hệ: N actions — 1 devices (một thiết bị có nhiều lượt thao tác); N actions — 1 users (một người dùng có thể thực hiện nhiều thao tác).

II. Giao diện



Hình 3: Giao diện Dashboard

ThanhIoT

Dashboard

Datasensor

History

Profile

←

Dữ liệu cảm biến

Q

Tìm kiếm theo thời gian

Loại cảm biến: Nhiệt độ

Đặt lại

ID	Loại cảm biến	Giá trị cảm biến	Thời gian đo
15	Nhiệt độ	32.4°C	15/08/2026 22:11:31
14	Nhiệt độ	32.4°C	15/08/2026 22:11:31
13	Nhiệt độ	33.1°C	15/08/2026 22:11:31
12	Nhiệt độ	33.1°C	15/08/2026 22:11:29
11	Nhiệt độ	29.8°C	15/08/2026 22:11:29
10	Nhiệt độ	29.8°C	15/08/2026 22:11:29
9	Nhiệt độ	29.8°C	15/08/2026 22:11:27

Hiển thị 1 đến 7 trong 15 bản ghi

Số hàng: 7

Trang 1/3

<

>

Hình 4: Giao diện Datasensors

ThanhIoT

Dashboard

Datasensor

History

Profile

←

Lịch sử bật tắt thiết bị

Q

Tìm kiếm theo thời gian

Tên thiết bị: LED 1

Hành động: Bật

Trạng thái: Thành công

Đặt lại

ID	Người thực hiện	Tên thiết bị	Hành động	Trạng thái	Thời gian thực hiện
15	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:31
14	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:31
13	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:31
12	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:29
11	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:29
10	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:29
9	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:27

Hiển thị 1 đến 7 trong 15 bản ghi

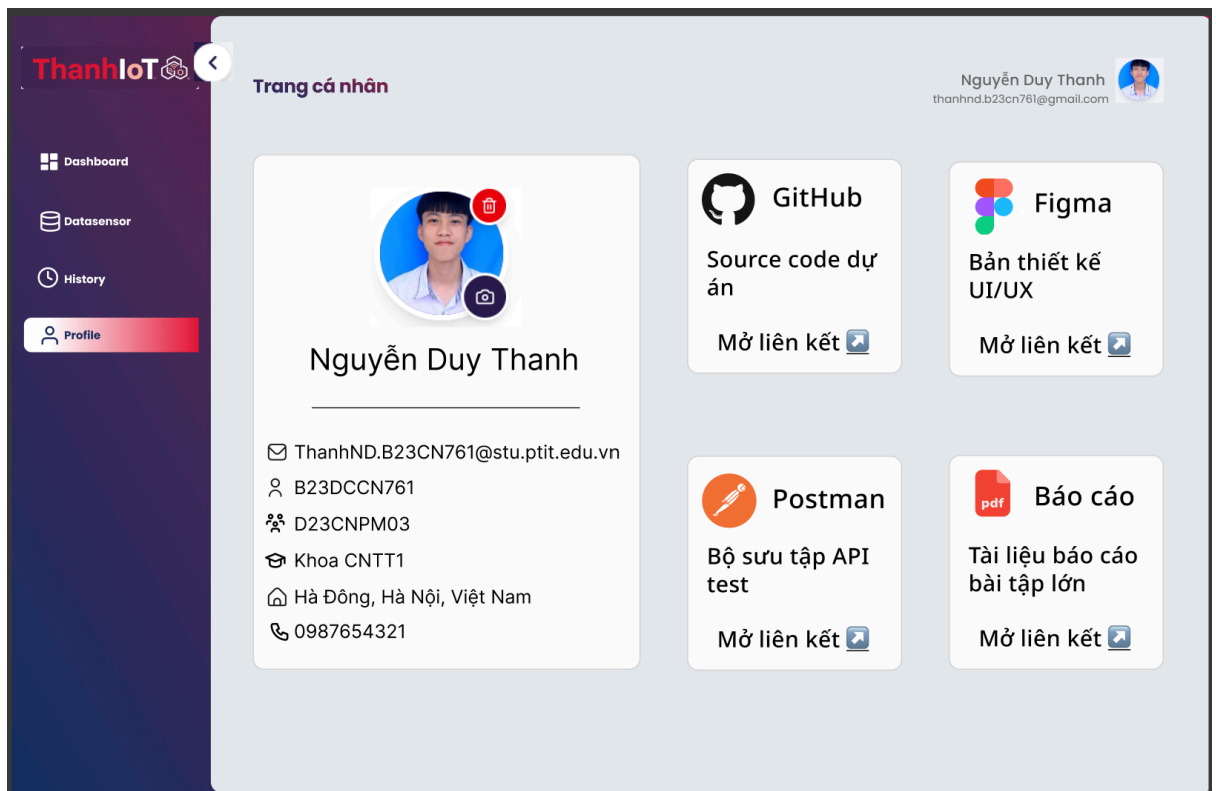
Số hàng: 7

Trang 1/3

<

>

Hình 5: Giao diện trang History

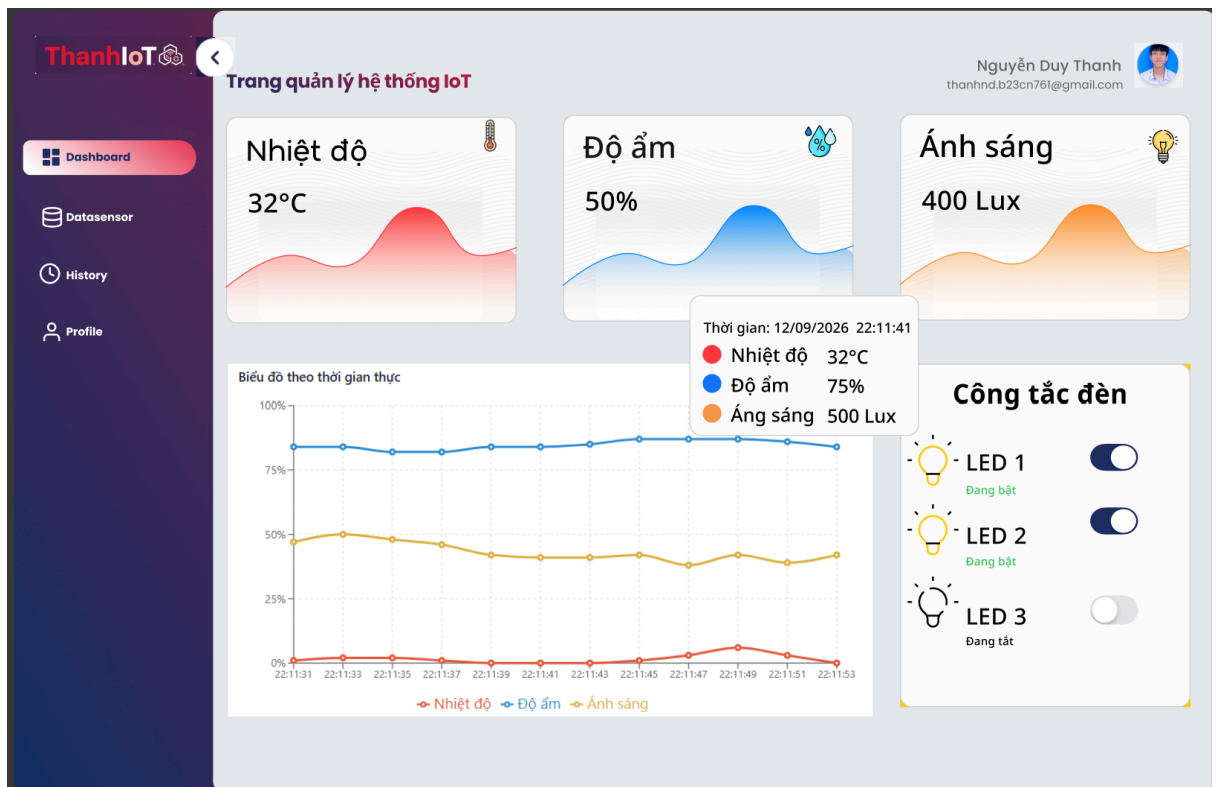


Hình 6: Giao diện trang Profile

III. Use case

A. Use case 1: Theo dõi dữ liệu cảm biến

1. Giao diện



Hình 7: Giao diện theo dõi cảm biến realtime

2. Actor

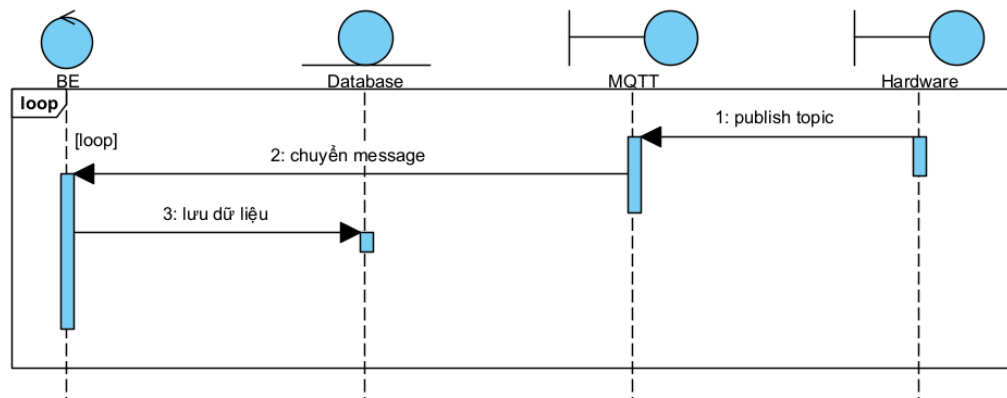
- Actor chính: User — truy cập giao diện Dashboard để theo dõi các giá trị cảm biến (nhiệt độ, độ ẩm, ánh sáng) theo thời gian thực, dưới cả 2 hình thức: giá trị hiện tại và biểu đồ biến thiên theo thời gian.
- Các thành phần: User, Frontend, Backend, Database, MQTT Broker, Hardware.

3. Mô tả use case

- Use case cho phép User theo dõi dữ liệu cảm biến real-time trên Dashboard, được thể hiện qua 2 dạng hiển thị trên cùng một màn hình:
 - + Giá trị hiện tại (dạng số): Nhiệt độ (°C), độ ẩm (%), ánh sáng (Lux).
 - + Biểu đồ biến thiên: Cả 3 giá trị theo thời gian, quy đổi về thang 0–100% ở Frontend để hiển thị chung trên một biểu đồ, hiển thị theo 12 mốc thời gian gần nhất (khởi tạo 12 mốc từ API và duy trì liên tục qua WebSocket).
 - + Công thức chuẩn hoá thang đo 0–100% tại Frontend:
 - + Cảm biến DHT11 có dải đo hoạt động từ 0°C đến 50°C. Giá trị nhiệt độ thực tế thu thập được sẽ được chia cho ngưỡng cực đại (50°C) rồi nhân với 100 để quy đổi về thang đo phần trăm.
 - + Do giá trị độ ẩm từ cảm biến vốn đã đo theo tỷ lệ phần trăm từ 0% đến 100%, giá trị này được giữ nguyên mà không cần qua bước biến đổi toán học.
 - + Tín hiệu từ cảm biến ánh sáng (quang trở) được đọc qua bộ chuyển đổi tương tự - số (ADC 10-bit) trên chân A0 của vi điều khiển ESP8266, trả về dải giá trị số nguyên từ 0 đến 1023. Giá trị này được chuẩn hóa về thang phần trăm bằng cách chia cho giá trị cực đại 1023 và nhân với 100, kết hợp hàm giới hạn mức tối đa ở 100%.
- Cả 2 dạng hiển thị đều dùng chung một cơ chế cập nhật dữ liệu:
 - + Hardware định kỳ 2 giây/lần đọc giá trị cảm biến, publish lên topic MQTT `iot/sensors/data`.
 - + Backend subscribe topic tương ứng, nhận dữ liệu định kỳ và lưu vào Database.
 - + Frontend gọi REST API để lấy dữ liệu ban đầu khi vừa vào Dashboard, sau đó nhận cập nhật liên tục qua WebSocket (STOMP Broker) mà không cần polling lại API.

4. Biểu đồ tuần tự

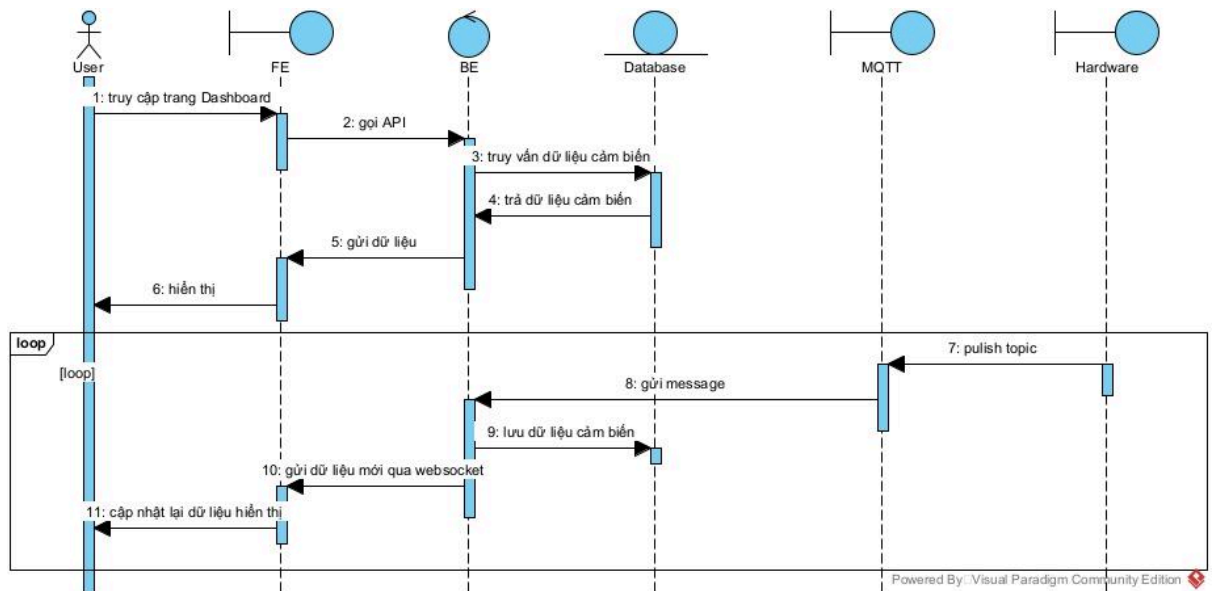
Biểu đồ 1: Luồng hệ thống nhận dữ liệu từ Hardware



Hình 8: Biểu đồ tuần tự luồng hệ thống nhận dữ liệu cảm biến

- Bước 1: Hardware ESP8266 đo nhiệt độ, độ ẩm, ánh sáng, sau đó gom 3 giá trị lại trong một message rồi publish lên topic “iot/sensors/data” của MQTT Broker Mosquitto.
- Bước 2: Mosquitto chuyển message qua Backend vốn đã subscribe topic “iot/sensors/data” từ trước đó.
- Bước 3: Backend sau khi nhận message, thực hiện chuẩn hóa dữ liệu rồi lưu vào cơ sở dữ liệu.
- Quá trình trên được thực hiện định kỳ mỗi 2 giây một lần.

Biểu đồ 2: Luồng user xem cảm biến realtime



Hình 9: Biểu đồ tuần tự luồng user xem giá trị cảm biến

a. Flow A: User xem giá trị hiện tại

- Bước 1: User truy cập trang Dashboard.
- Bước 2: Frontend khi này gọi API GET /api/datasensors/latest để lấy bản ghi mới nhất của 3 giá trị nhiệt độ, độ ẩm, ánh sáng.
- Bước 3: Backend khi này thực hiện truy vấn cơ sở dữ liệu để lấy bản ghi mới nhất của từng loại cảm biến.
- Bước 4: Cơ sở dữ liệu trả dữ liệu về Backend.
- Bước 5: Backend gửi dữ liệu về Frontend.
- Bước 6: Frontend thực hiện hiển thị dữ liệu lên giao diện cho User.
- Bước 7: Hardware ESP8266 định kỳ đo nhiệt độ, độ ẩm, ánh sáng, sau đó gom 3 giá trị lại trong một message rồi publish lên topic “iot/sensors/data” của MQTT Broker Mosquitto.
- Bước 8: Mosquitto chuyển message qua Backend vốn đã subscribe topic “iot/sensors/data” từ trước đó.
- Bước 9: Backend sau khi nhận message, thực hiện chuẩn hóa dữ liệu rồi lưu vào cơ sở dữ liệu.
- Bước 10: Backend gửi dữ liệu mới về Frontend thông qua WebSocket.
- Bước 11: Frontend cập nhật dữ liệu hiển thị mới trên giao diện.
- Quá trình từ bước 7 đến bước 11 sẽ được lặp lại định kỳ để liên tục hiển thị dữ liệu cảm biến mới nhất đến người dùng.

b. Flow B: User xem biểu đồ

- Bước 1: User truy cập trang Dashboard để xem biểu đồ.
- Bước 2: Frontend khi này gọi API GET /api/datasensors/chart/latest để lấy 12 bản ghi mới nhất của 3 giá trị nhiệt độ, độ ẩm, ánh sáng.
- Bước 3: Backend khi này thực hiện truy vấn cơ sở dữ liệu để lấy 12 mốc dữ liệu gần nhất, mỗi mốc gồm nhiệt độ, độ ẩm và ánh sáng. .
- Bước 4: Cơ sở dữ liệu trả dữ liệu về Backend.
- Bước 5: Backend gửi dữ liệu về Frontend.
- Bước 6: Frontend quy đổi các giá trị về thang 100% sau đó vẽ biểu đồ và hiển thị dữ liệu lên giao diện cho User.
- Bước 7: Hardware ESP8266 định kỳ đo nhiệt độ, độ ẩm, ánh sáng, sau đó gom 3 giá trị lại trong một message rồi publish lên topic “iot/sensors/data” của MQTT Broker Mosquitto.
- Bước 8: Mosquitto chuyển message qua Backend vốn đã subscribe topic “iot/sensors/data” từ trước đó.

- Bước 9: Backend sau khi nhận message, thực hiện chuẩn hóa dữ liệu rồi lưu vào cơ sở dữ liệu.
- Bước 10: Backend gửi dữ liệu mới về Frontend thông qua WebSocket.
- Bước 11: Frontend chuẩn hoá điểm mới, thêm vào cuối biểu đồ, xóa điểm cũ nhất để duy trì 12 điểm gần nhất và cập nhật lại biểu đồ.
- Quá trình từ bước 7 đến bước 11 sẽ được lặp lại định kỳ để liên tục hiển thị dữ liệu cảm biến mới nhất đến người dùng.

5. *API documents*

5.1. *API lấy dữ liệu cảm biến mới nhất*

- Method: GET
- Endpoint: /api/datasensors/latest
- Mục đích: Lấy giá trị mới nhất của nhiệt độ, độ ẩm và ánh sáng để hiển thị khi User vừa truy cập Dashboard.
- Response:


```
{
  "success": true,
  "message": "Sensors value fetched successfully!",
  "data": {
    "temperature": 28.5,
    "humidity": 72.3,
    "light": 650,
    "timestamp": "2026-08-18T10:00:00"
  }
}
```

5.2. *API lấy dữ liệu ban đầu cho biểu đồ*

- Method: GET
- Endpoint: /api/datasensors/chart/latest
- Mục đích: Lấy 12 mốc dữ liệu cảm biến gần nhất để khởi tạo biểu đồ.
- Response:


```
{
  "success": true,
  "message": "Chart data fetched successfully!",
  "data": [
    {
      "timestamp": "2026-08-18T10:00:00",
      "temperature": 28.5,
```

```

        "humidity": 72.3,
        "light": 650
    },
    {
        "timestamp": "2026-08-18T10:00:05",
        "temperature": 28.7,
        "humidity": 72.1,
        "light": 670
    },
    {
        "timestamp": "2026-08-18T10:00:10",
        "temperature": 28.9,
        "humidity": 71.9,
        "light": 680
    }
]
}

```

5.3. Luồng dữ liệu realtime qua MQTT

- MQTT Topic: `iot/sensors/data`.
- Publisher: ESP8266
- Subscriber: Backend
- Mục đích: Nhận dữ liệu nhiệt độ, độ ẩm và ánh sáng từ ESP8266 theo chu kỳ định kỳ.
- Message từ ESP8266:


```

{
    "temperature": 28.7,
    "humidity": 72.1,
    "light": 670
}

```

5.4. WebSocket cập nhật dữ liệu realtime

- WebSocket endpoint: `/ws/sensors`
- Channel (Topic) subscribe: `/topic/sensors`
- Giao thức: STOMP qua SockJS
- Mục đích: Gửi dữ liệu cảm biến mới từ BE đến FE mà không cần FE liên tục gọi REST API.
- Message Backend gửi cho Frontend:


```

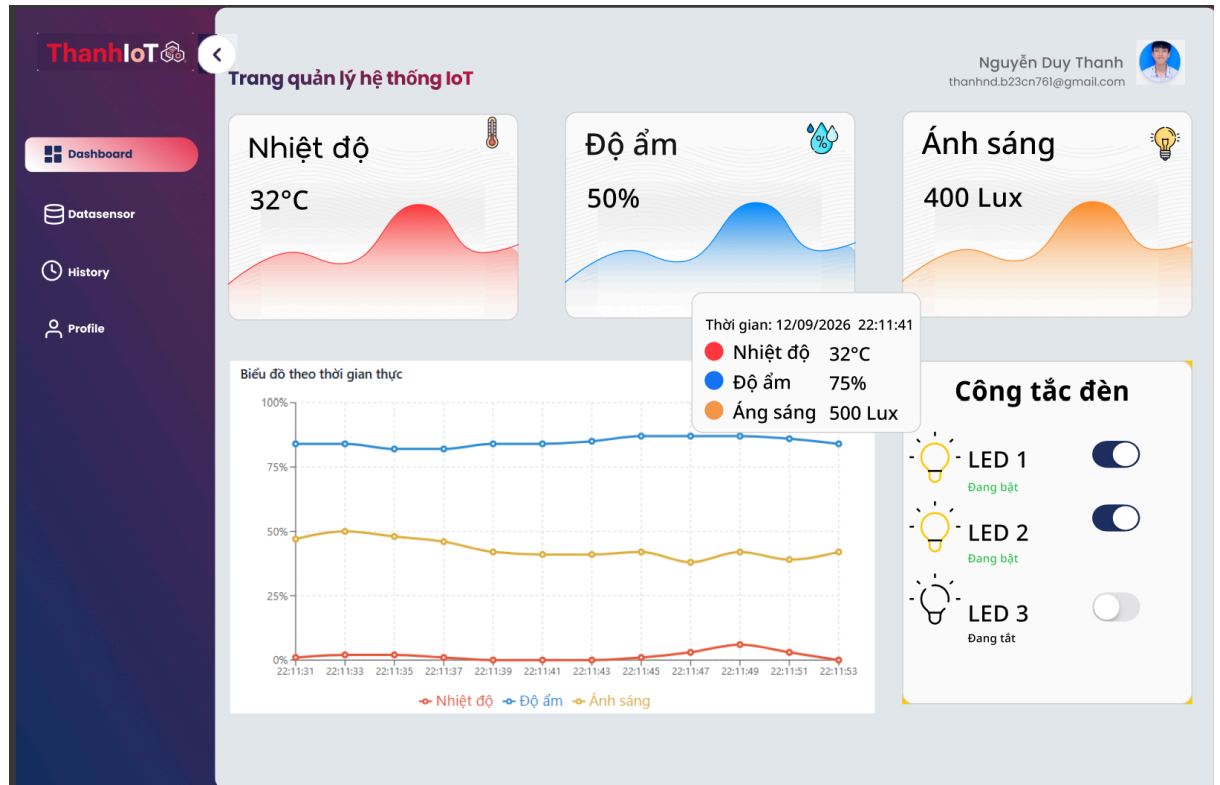
{
    "temperature": 28.7,

```

```
"humidity": 72.1,  
"light": 670,  
"timestamp": "2026-08-18T10:00:05"  
}
```

B. Use case 2: Bật tắt đèn LED

1. Giao diện



Hình 10: Giao diện điều khiển đèn

2. Actor

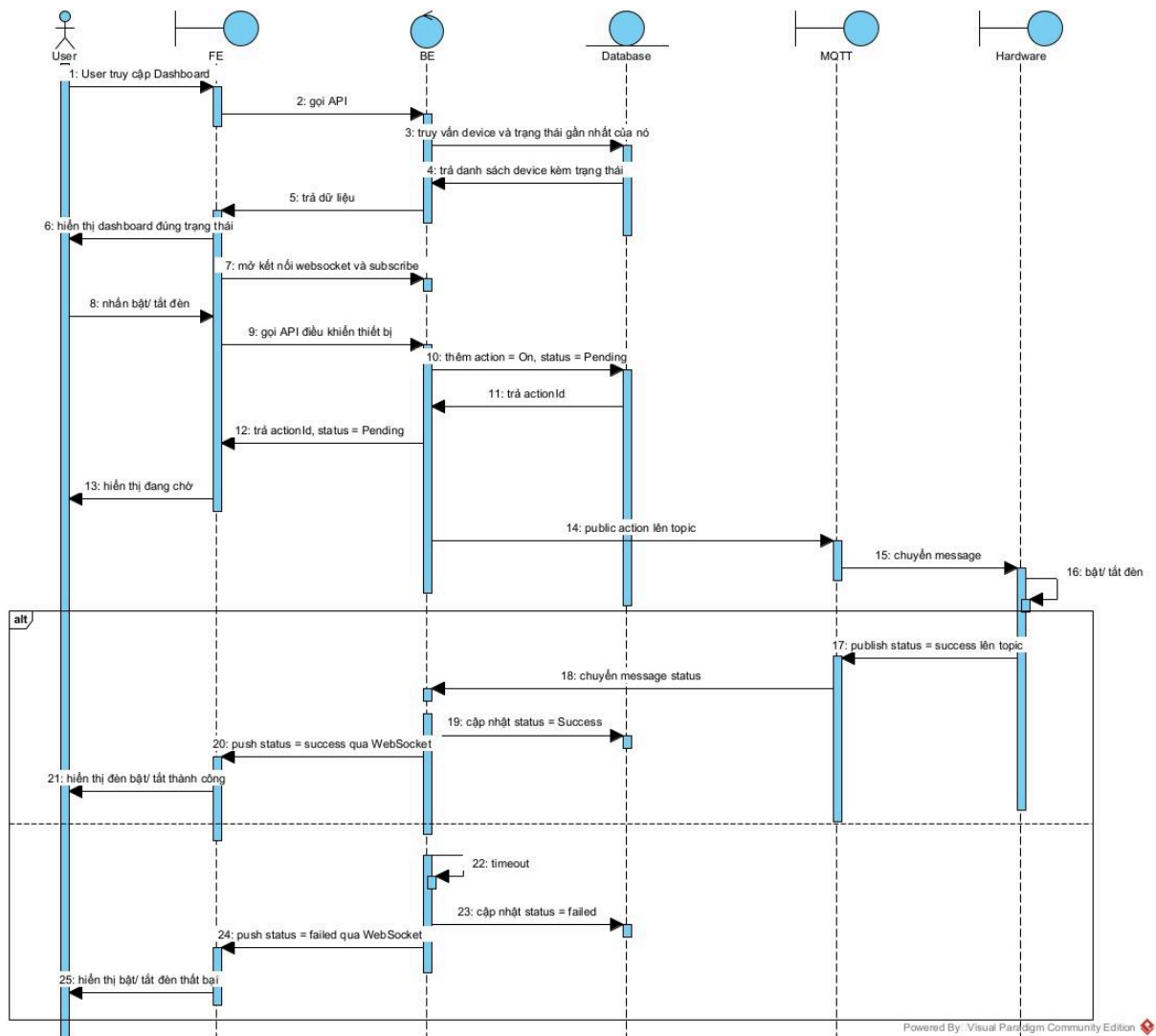
- Actor chính: User
- User có thể bật/ tắt đèn Led.
- Các thành phần bao gồm: User, Frontend, Backend, Database, MQTT, Hardware.

3. Mô tả usecase

- Khi User gạt nút ON/OFF trên Dashboard:
 - + Frontend cập nhật trạng thái chờ (Pending) và gửi request REST API đến Backend.
 - + Backend tạo một bản ghi trong bảng actions với trạng thái ban đầu là PENDING.
 - + Backend publish một message chứa thông tin action đến topic điều khiển trên MQTT.
 - + ESP8266 nhận message và điều khiển chân GPIO của LED tương ứng.

- + Sau khi thực hiện xong, ESP8266 gửi message chứa thông tin trạng thái trở lại MQTT Broker.
- + Backend nhận message trạng thái từ MQTT và cập nhật actions.status thành SUCCESS (nếu thành công) hoặc FAILED (nếu lỗi).
- + Backend đẩy kết quả qua kênh WebSocket riêng của thiết bị để Frontend cập nhật trạng thái đèn trên UI.
- + Cơ chế Timeout: Nếu sau 15 giây mà không nhận được phản hồi từ Hardware, Backend tự động đánh dấu action là FAILED và thông báo lỗi qua WebSocket.

4. Biểu đồ tuần tự



Hình 11: Biểu đồ tuần tự luồng điều khiển thiết bị

- Bước 1: User truy cập Dashboard.
- Bước 2: Frontend gọi API GET /api/devices để lấy danh sách thiết bị cùng trạng thái gần nhất của chúng.
- Bước 3: Backend nhận request, truy vấn dữ liệu ở Database.

- Bước 4: Database trả dữ liệu cho backend.
- Bước 5: Backend trả dữ liệu cho Frontend.
- Bước 6: Frontend hiển thị danh sách device đúng với trạng thái của chúng.
- Bước 7: Frontend mở kết nối WebSocket với Backend và subscribe kênh cập nhật trạng thái thiết bị.
- Bước 8: User gạt nút bật đèn LED.
- Bước 9: Frontend gọi API POST `/api/devices/{deviceId}/actions` đến Backend.
- Bước 10: Backend tạo một bản ghi mới trong bảng actions, với action là hành động User yêu cầu và status = Pending.
- Bước 11: Database trả lại actionId cho Backend theo dõi.
- Bước 12: Backend trả actionId cùng trạng thái Pending cho Frontend.
- Bước 13: Frontend hiển thị thông báo đang chờ bật/ tắt thiết bị.
- Bước 14: Backend tạo MQTT message chứa actionId, deviceId và action và publish lên topic `iot/devices/{deviceId}/action`.
- Bước 15: MQTT chuyển message tới Hardware đang subscribe topic tương ứng.
- Bước 16: Hardware bật/ tắt đèn LED mà User mong muốn.
- Bước 17: Sau khi thực hiện việc bật/ tắt đèn, Hardware gửi message chứa trạng thái đèn trở lại MQTT trong topic `iot/devices/{deviceId}/status`.
- Bước 18: MQTT chuyển message tới Backend đang subscribe topic tương ứng.
- Bước 19: Backend chuẩn hoá dữ liệu từ message và cập nhật lại trạng thái của actionId trong Database.
- Bước 20: Backend publish trạng thái cuối cùng lên WebSocket channel mà Frontend đã subscribe.
- Bước 21: Frontend cập nhật lại trạng thái của đèn LED trên giao diện.
- Bước 22: Trong trường hợp Hardware lỗi không thực hiện được action, Backend sẽ bị quá hạn timeout sau 15 giây.
- Bước 23: Backend cập nhật trạng thái của actionId trong Database là Failed.
- Bước 24: Backend publish trạng thái cuối cùng lên WebSocket channel mà Frontend đã subscribe.
- Bước 25: Frontend thông báo bật/ tắt đèn thất bại trên giao diện.

5. *API documents*

5.1. *API lấy danh sách thiết bị ban đầu*

- Method: GET
- Endpoint: /api/devices
- Mục đích: Lấy danh sách 3 đèn LED cùng trạng thái gần nhất (lastAction: ON/OFF) khi vừa mở Dashboard.
- Response:

```
{
  "success": true,
  "message": "Devices fetched successfully!",
  "data": [
    {
      "id": 1,
      "name": "LED 1",
      "lastAction": "ON",
      "status": "SUCCESS",
      "lastUpdatedAt": "2026-08-18T10:00:00"
    },
    {
      "id": 2,
      "name": "LED 2",
      "lastAction": "OFF",
      "status": "SUCCESS",
      "lastUpdatedAt": "2026-08-18T09:45:00"
    },
    {
      "id": 3,
      "name": "LED 3",
      "lastAction": "OFF",
      "status": "SUCCESS",
      "lastUpdatedAt": "2026-08-18T09:30:00"
    }
  ]
}
```

5.2. *API gửi yêu cầu bật/tắt thiết bị*

- Method: POST
- Endpoint: /api/devices/{deviceId}/actions

- Mục đích: Gửi yêu cầu bật hoặc tắt một đèn LED.

- Request:

```
{  
  "userId": 101,  
  "action": "ON"  
}
```

- Response:

```
{  
  "success": true,  
  "message": "Action accepted",  
  "data": {  
    "actionId": 15,  
    "deviceId": 1,  
    "userId": 101,  
    "action": "ON",  
    "status": "PENDING"  
  }  
}
```

5.3. MQTT API – Gửi lệnh điều khiển

- Topic: `iot/devices/{deviceId}/action`
- Publisher: Backend
- Subscriber: Hardware
- Message:

```
{  
  "actionId": 15,  
  "deviceId": 1,  
  "action": "ON"  
}
```

- Mô tả: Backend publish message lên MQTT Broker. MQTT Broker chuyển message đến ESP8266. ESP8266 đọc trường action và thực hiện thao tác tương ứng.

5.4. MQTT API – Nhận kết quả thực hiện

- Topic: `iot/devices/{deviceId}/status`
- Publisher: Hardware
- Subscriber: Backend
- Message:

```
{  
  "actionId": 15,  
  "deviceId": 1,
```

```
"action": "ON",  
"status": "SUCCESS/ FAILED"  
}
```

5.5. WebSocket API – Đẩy kết quả cuối cùng về giao diện

- Endpoint kết nối: /ws/devices.
- Channel subscribe: /topic/devices/{deviceId}/status.
- Publisher: Backend
- Subscriber: Frontend
- Message:

```
{  
  "actionId": 15,  
  "deviceId": 1,  
  "userId": 101,  
  "action": "ON",  
  "status": "SUCCESS"  
}
```
- Mô tả: Ngay sau khi Backend xác định được kết quả cuối cùng của một action, hoặc do nhận được phản hồi thất từ Hardware qua MQTT, hoặc do tự phát hiện quá thời gian timeout mà không có phản hồi, Backend sẽ publish message này lên channel WebSocket tương ứng. Frontend đang subscribe sẵn sẽ nhận ngay lập tức và cập nhật giao diện.

C. Use case 3: Tra cứu lịch sử

1. Giao diện

ThanhIoT

Dữ liệu cảm biến

Tìm kiếm theo thời gian

Loại cảm biến: Nhiệt độ

Đặt lại

ID	Loại cảm biến	Giá trị cảm biến	Thời gian đo
15	Nhiệt độ	32.4°C	15/08/2026 22:11:31
14	Nhiệt độ	32.4°C	15/08/2026 22:11:31
13	Nhiệt độ	33.1°C	15/08/2026 22:11:31
12	Nhiệt độ	33.1°C	15/08/2026 22:11:29
11	Nhiệt độ	29.8°C	15/08/2026 22:11:29
10	Nhiệt độ	29.8°C	15/08/2026 22:11:29
9	Nhiệt độ	29.8°C	15/08/2026 22:11:27

Hiển thị 1 đến 7 trong 15 bản ghi

Số hàng: 7 Trang 1/3

Hình 12: Giao diện trang lịch sử cảm biến

ThanhIoT

Lịch sử bật tắt thiết bị

Tìm kiếm theo thời gian

Tên thiết bị: LED 1

Hành động: Bật

Trạng thái: Thành công

Đặt lại

ID	Người thực hiện	Tên thiết bị	Hành động	Trạng thái	Thời gian thực hiện
15	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:31
14	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:31
13	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:31
12	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:29
11	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:29
10	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:29
9	Nguyễn Duy Thanh	LED 1	Bật	Thành công	15/08/2026 22:11:27

Hiển thị 1 đến 7 trong 15 bản ghi

Số hàng: 7 Trang 1/3

Hình 13: Giao diện trang lịch sử điều khiển thiết bị

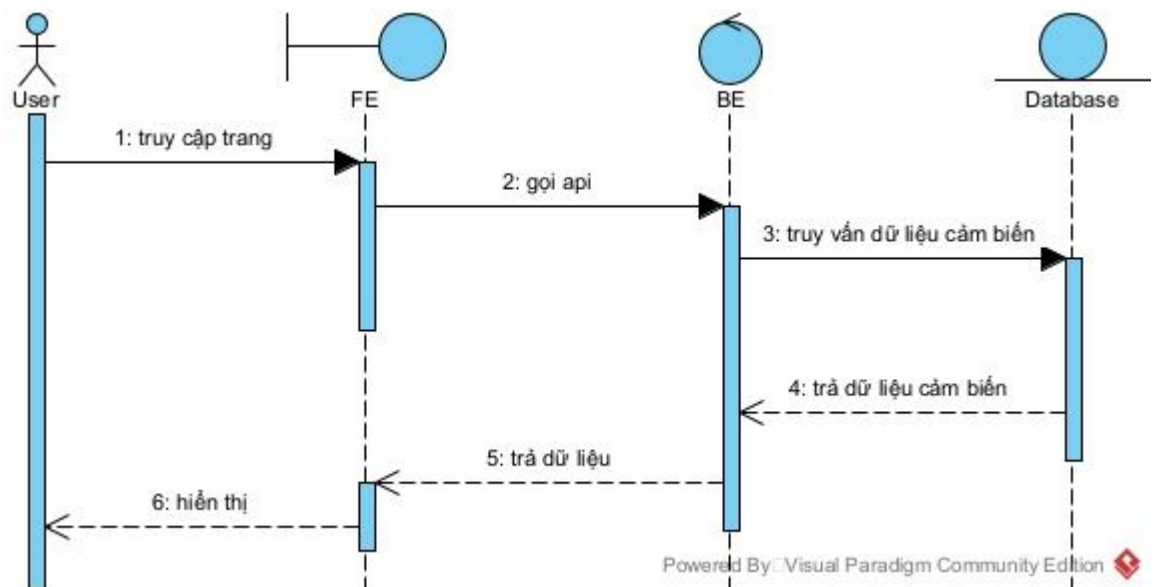
2. Actor

- Actor chính: User — tra cứu lại dữ liệu lịch sử mà hệ thống đã ghi nhận, gồm cả lịch sử dữ liệu cảm biến và lịch sử thao tác bật/tắt thiết bị.
- Các thành phần: User, Frontend, Backend, Database.

3. Mô tả usecase

- Đây là một mục tiêu duy nhất của User ("tra cứu lại dữ liệu lịch sử"), áp dụng trên 2 nguồn dữ liệu khác nhau nhưng theo cùng một khuôn mẫu thao tác: xem dưới dạng bảng, hỗ trợ phân trang, lọc và sắp xếp theo thời gian.
 - + Tra cứu lịch sử cảm biến: xem lại dữ liệu nhiệt độ/độ ẩm/ánh sáng đã thu thập, tìm kiếm theo thời gian hoặc theo loại cảm biến.
 - + Tra cứu lịch sử bật tắt: xem lại các thao tác bật/tắt LED đã thực hiện, lọc theo tên thiết bị, tìm kiếm theo thời gian.

4. Biểu đồ tuần tự



Hình 14: Biểu đồ tuần tự luồng tra cứu lịch sử

- Bước 1: User truy cập trang tra cứu (trang Datasensor hoặc trang History).
- Bước 2: Frontend gọi API tương ứng đến Backend, kèm tham số phân trang/lọc/tìm kiếm.
- Bước 3: Backend tạo truy vấn dựa trên các tham số lọc và phân trang rồi gửi tới Database.
- Bước 4: Database trả về tập dữ liệu tương ứng với điều kiện truy vấn.
- Bước 5: Backend đóng gói dữ liệu và gửi về Frontend.
- Bước 6: Frontend xử lý dữ liệu và hiển thị thành bảng trên giao diện.

5. API document

5.1. API lấy danh sách dữ liệu cảm biến

- Method: GET
- Endpoint: /api/datasensors
- Mục đích: Lấy danh sách các bản ghi dữ liệu cảm biến để hiển thị trên trang Datasensor.
- Database: sensors, datasensors.
- Query parameters:
 - + page: Số trang cần lấy, bắt đầu từ 0
 - + size: Số bản ghi hiển thị trong 1 trang.
 - + type: Loại cảm biến cần lọc, bao gồm All, Temperature , Humidity, Light.
 - + search: Từ khoá tìm kiếm linh hoạt (theo giá trị đo hoặc thời gian).
 - + searchMode: Chế độ tìm kiếm (ALL - tìm cả giá trị & thời gian; TIME - chỉ tìm thời gian).
 - + time: Mốc thời gian tìm kiếm.
- Ví dụ: GET /api/datasensors?page=0&size=10&type=Temperature
 - + Lấy trang đầu tiên.
 - + Mỗi trang chứa 10 bản ghi.
 - + Hiển thị bản ghi của cảm biến nhiệt độ.
- Ví dụ: GET /api/datasensors?time=2026-08-18
 - + Tìm kiếm các giá trị cảm biến trong ngày 18/8/2026.
- Ví dụ response Backend trả về:

```
{
  "success": true,
  "message": "Datasensors fetched successfully!",
  "data": {
    "content": [
      {
        "id": 105,
        "sensorId": 1,
        "sensorType": "temperature",
        "value": 28.7,
        "createdAt": "2026-08-18T11:59:50"
      },
      {
        "id": 104,
```

```

        "sensorId": 1,
        "sensorType": "temperature",
        "value": 28.5,
        "createdAt": "2026-08-18T11:59:40"
    },
    {
        "id": 103,
        "sensorId": 1,
        "sensorType": "temperature",
        "value": 28.4,
        "createdAt": "2026-08-18T11:59:30"
    }
],
"page": 0,
"size": 10,
"totalElements": 35,
"totalPages": 4
}
}

```

5.2. API lấy lịch sử bật/tắt

- Method: GET
- Endpoint: /api/actions
- Mục đích: Lấy danh sách lịch sử các thao tác bật/tắt thiết bị để hiển thị trên giao diện.
- Database: devices, actions.
- Query parameters:
 - + page: Số trang cần lấy, bắt đầu từ 0
 - + size: Số bản ghi hiển thị trong 1 trang.
 - + device: Tên thiết bị, bao gồm 3 đèn LED.
 - + status: Trạng thái điều khiển thiết bị.
 - + action: Lọc theo hành động (ON hoặc OFF).
 - + time: Mốc thời gian tìm kiếm.
- Ví dụ: GET /api/actions?page=0&size=10&device=LED%201
 - + Lấy lịch sử thao tác của đèn LED 1, phân thành nhiều trang mỗi trang 10 bản ghi, bắt đầu hiển thị từ trang 1.
- Ví dụ response:


```

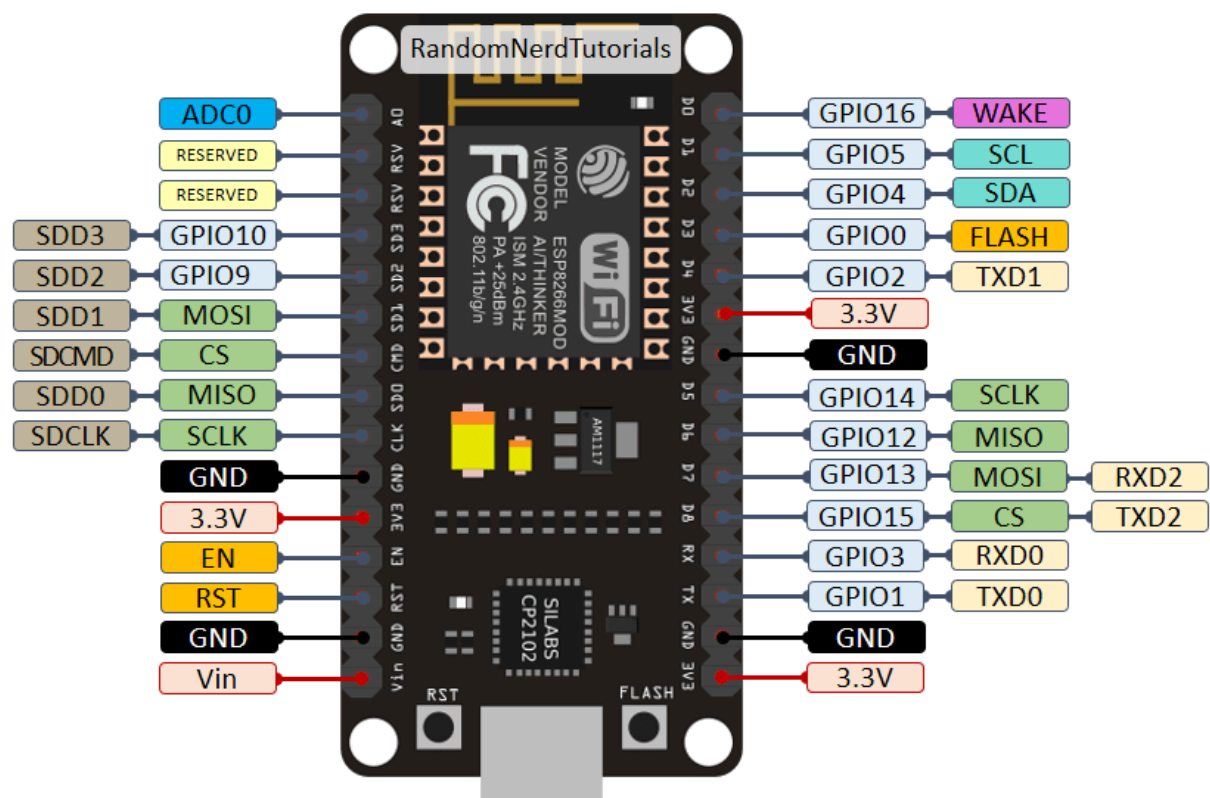
{
    "success": true,
    "message": "Actions fetched successfully!",

```

```
"data": {
  "content": [
    {
      "id": 15,
      "deviceId": 1,
      "deviceName": "LED 1",
      "userId": 1,
      "action": "ON",
      "status": "SUCCESS",
      "createdAt": "2026-08-18T11:59:50"
    },
    {
      "id": 14,
      "deviceId": 1,
      "deviceName": "LED 1",
      "userId": 1,
      "action": "OFF",
      "status": "SUCCESS",
      "createdAt": "2026-08-18T11:58:30"
    },
    {
      "id": 13,
      "deviceId": 1,
      "deviceName": "LED 1",
      "userId": 1,
      "action": "ON",
      "status": "FAILED",
      "createdAt": "2026-08-18T11:55:10"
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 35,
  "totalPages": 4
}
```


CHƯƠNG 4: THIẾT KẾ VÀ CÀI ĐẶT PHẦN CỨNG

I. Thiết kế phần cứng



Hình 15: Sơ đồ chân bộ kit ESP8266 12-E NodeMCU

1. Bảng quy hoạch chân kết nối

Khôì/ thiết bị	Chân trên linh kiện	Vị trí kết nối	Mô tả chức năng
Nguồn và bo mạch	Chân 3V (ESP8266)	Đường ray Đỏ (+)	Trích xuất nguồn 3.3V cấp cho toàn bộ mạch
	Chân G (ESP8266)	Đường ray Xanh (-)	Nối Mass (GND) cấp chung cho toàn hệ thống
	Cáp Jumper (Cầu nối)	Đỏ nối Đỏ, Xanh nối Xanh	Đồng bộ nguồn điện VCC và GND giữa 2 Breadboard
Cảm biến DHT11	Chân + (VCC)	Đường ray Đỏ (+)	Nhận điện áp 3.3V từ

			Breadboard
	Chân - (GND)	Đường ray Xanh (-)	Nối chung về Mass (GND)
	Chân OUT	Chân D1 (ESP8266)	Truyền dữ liệu tín hiệu số (Digital)
Module Ánh sáng	Chân VCC	Đường ray Đỏ (+)	Nhận điện áp 3.3V từ Breadboard
	Chân GND	Đường ray Xanh (-)	Nối chung về Mass (GND)
	Chân DO	Chân A0 (ESP8266)	Truyền dữ liệu tín hiệu tương tự (Analog)
Khối Hiển thị (LED)	LED 1 (Chân dài/Anode)	Nối tiếp điện trở đến chân D5	Nhận tín hiệu điều khiển trạng thái 1
	LED 2 (Chân dài/Anode)	Nối tiếp điện trở đến chân D6	Nhận tín hiệu điều khiển trạng thái 2
	LED 3 (Chân dài/Anode)	Nối tiếp điện trở đến chân D7	Nhận tín hiệu điều khiển trạng thái 3
	Cụm 3 LED (Chân ngắn/Cathode)	Đường ray Xanh (-)	Xả dòng điện về Mass (GND) khép kín mạch

Bảng 6: Bảng quy hoạch chân kết nối

2. Diễn giải nguyên lý đấu nối

- Hệ thống sử dụng vi điều khiển NodeMCU ESP8266 làm bộ xử lý trung tâm. Nguồn điện hoạt động của toàn bộ mạch được trích xuất từ chân 3V (tương đương 3.3V) và chân G (GND) của vi điều khiển, sau đó phân bổ ra hai đường ray nguồn chính (đỏ và xanh) trên Breadboard. Để giải quyết bài toán về không gian lắp ráp, hệ thống thiết kế mở rộng trên hai Breadboard. Nguồn điện giữa hai bo mạch này được đồng bộ hóa thông qua cáp jumper liên kết trực tiếp các

đường ray cùng cực (Đỏ nối Đỏ, Xanh nối Xanh), đảm bảo duy trì tính ổn định của điện áp cấp cho các module ngoại vi.

- Tại khối thu thập dữ liệu, cảm biến nhiệt độ - độ ẩm DHT11 và module cảm biến ánh sáng được cấp nguồn từ các đường ray đồng bộ của hệ thống. Về luồng tín hiệu, dữ liệu môi trường từ chân OUT của DHT11 được truyền về vi điều khiển thông qua chân kỹ thuật số D1 (Digital). Trong khi đó, dữ liệu cường độ ánh sáng từ chân DO của cảm biến quang được đưa về chân A0 để xử lý qua bộ chuyển đổi tương tự - số (ADC).
- Tại khối cơ cấu chấp hành, ba đèn LED được thiết lập để làm nhiệm vụ cảnh báo và hiển thị trạng thái. Nhằm đảm bảo an toàn phần cứng và bảo vệ vi điều khiển khỏi hiện tượng quá dòng hoặc dòng dội ngược, cực dương (Anode) của mỗi LED đều được thiết kế mắc nối tiếp với một điện trở hạn dòng trước khi đấu nối vào các chân xuất tín hiệu điều khiển D5, D6 và D7. Cuối cùng, toàn bộ cực âm (Cathode) của hệ thống đèn LED được thiết kế chung chân và xả về đường ray Mass (GND) nhằm khép kín toàn bộ mạch điện.

II. Mã Nguồn Điều Khiển Thiết Bị (ESP8266)

1. Khai báo thư viện và cấu hình phần cứng

```

#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>
const char* ssid = "Thanh";
const char* password = "12345678";
const char* mqtt_server = "172.20.10.3";
const char* mqtt_user = "NguyenDuyThanh";
const char* mqtt_password = "B23DCCN761";
WiFiClient espClient;
PubSubClient client(espClient);
#define LED_1 D5
#define LED_2 D6
#define LED_3 D7
#define DHTPIN D1 // Chân OUT của DHT11
#define DHTTYPE DHT11 // Loại cảm biến DHT11
DHT dht(DHTPIN, DHTTYPE);

#define LDR_PIN A0 // Chân A0 của cảm biến ánh sáng

```

Hình 16: Mã nguồn khai báo thư viện và phần cứng

- Thư viện: Chương trình sử dụng ESP8266WiFi để kết nối mạng, PubSubClient để giao tiếp qua giao thức MQTT và ArduinoJson để chuẩn hóa dữ liệu giao tiếp dưới định dạng JSON, giúp Backend dễ dàng phân giải.
- Phần cứng: 3 thiết bị chấp hành (LED) được điều khiển qua các chân Digital (D5, D6, D7). Cảm biến DHT11 truyền dữ liệu số qua chân D1, và cảm biến ánh sáng (LDR) sử dụng chân Analog A0 để đọc giá trị điện áp liên tục.

2. Cài đặt và khởi tạo hệ thống

```
void setup_wifi() {  
    delay(10);  
    Serial.println();  
    Serial.print("Đang kết nối WiFi: ");  
    Serial.println(ssid);  
    WiFi.begin(ssid, password);  
    while (WiFi.status() != WL_CONNECTED) {  
        delay(500);  
        Serial.print(".");  
    }  
    Serial.println("\nWiFi đã kết nối thành công!");  
    Serial.print("Địa chỉ IP của mạch: ");  
    Serial.println(WiFi.localIP());  
}
```

Hình 17: Mã nguồn hàm cài đặt wifi

```
void setup() {  
    Serial.begin(115200);  
    randomSeed(micros());  
  
    pinMode(LED_1, OUTPUT);  
    pinMode(LED_2, OUTPUT);  
    pinMode(LED_3, OUTPUT);  
    digitalWrite(LED_1, LOW);  
    digitalWrite(LED_2, LOW);  
    digitalWrite(LED_3, LOW);  
  
    dht.begin(); // Khởi động cảm biến DHT11  
  
    setup_wifi();  
    client.setServer(mqtt_server, 1884);  
    client.setCallback(callback);  
}
```

Hình 18: Mã nguồn hàm khởi tạo hệ thống

- Các chân điều khiển LED được cấu hình là OUTPUT và ngay lập tức được ghi mức logic LOW. Đây là thiết kế an toàn (Fail-safe) để tránh tình trạng các thiết bị ngoại vi tự động bật khi mạch vừa khởi động lại do sự cố.
- Hệ thống thiết lập kết nối tới MQTT Broker ở cổng mặc định 1884 và đăng ký hàm callback – một ngắt phần mềm sẽ tự động kích hoạt mỗi khi có bản tin MQTT từ Backend gửi xuống.

3. Cơ chế duy trì kết nối MQTT

```

void reconnect() {
    while (!client.connected()) {
        Serial.print("Đang kết nối MQTT Broker...");
        String clientId = "ESP8266Client-";
        clientId += String(random(0xffff), HEX);

        if (client.connect(clientId.c_str(), mqtt_user, mqtt_password)) {
            Serial.println("Thành công!");
            client.subscribe("iot/devices/+/action");
        } else {
            Serial.print("Thất bại, mã lỗi: ");
            Serial.print(client.state());
            Serial.println(" Thử lại sau 5 giây...");
            delay(5000);
        }
    }
}

```

Hình 19: Mã nguồn hàm duy trì kết nối

- Quản lý Client ID: Mã nguồn tạo clientId ngẫu nhiên để tránh xung đột (Conflict) nếu có nhiều thiết bị ESP8266 cùng kết nối vào một Broker.
- Tối ưu Subscribe: Thay vì subscribe từng topic riêng lẻ, hệ thống sử dụng ký tự đại diện Wildcard + trong topic iot/devices/+/action. Điều này giúp ESP8266 lắng nghe được lệnh điều khiển của tất cả các thiết bị trực thuộc nó chỉ với một lần đăng ký, tối ưu hóa bộ nhớ thiết bị.

4. Xử lý lệnh điều khiển và phản hồi chéo

```

void callback(char* topic, byte* payload, unsigned int length) {
    String message = "";

    for (int i = 0; i < length; i++) {
        message += (char)payload[i];
    }

    StaticJsonDocument<200> doc;
    DeserializationError error = deserializeJson(doc, message);

    if (error) return;

    int actionId = doc["actionId"];
    int deviceId = doc["deviceId"];
    String action = doc["action"];

    String statusResult = "FAILED";

    // deviceId = 0: điều khiển tất cả LED
    if (deviceId == 0) {
        if (action == "ON") {
            digitalWrite(LED_1, HIGH);
            digitalWrite(LED_2, HIGH);
            digitalWrite(LED_3, HIGH);

            if (digitalRead(LED_1) == HIGH && digitalRead(LED_2) == HIGH && digitalRead(LED_3) == HIGH) {
                statusResult = "SUCCESS";
            }
        } else if (action == "OFF") {
            digitalWrite(LED_1, LOW);
            digitalWrite(LED_2, LOW);
            digitalWrite(LED_3, LOW);

            if (digitalRead(LED_1) == LOW && digitalRead(LED_2) == LOW && digitalRead(LED_3) == LOW) {
                statusResult = "SUCCESS";
            }
        }
    }
}

```

```

// deviceId = 1, 2, 3: điều khiển từng LED
else {
    int targetPin = -1;

    if (deviceId == 1) {
        targetPin = LED_1;
    } else if (deviceId == 2) {
        targetPin = LED_2;
    } else if (deviceId == 3) {
        targetPin = LED_3;
    }
    if (targetPin != -1) {
        if (action == "ON") {
            digitalWrite(targetPin, HIGH);
        } else if (action == "OFF") {
            digitalWrite(targetPin, LOW);
        }
        int currentState = digitalRead(targetPin);
        if ((action == "ON" && currentState == HIGH) || (action == "OFF" && currentState == LOW)) {
            statusResult = "SUCCESS";
        }
    }
}

// Tạo JSON phản hồi
StaticJsonDocument<200> resDoc;

resDoc["actionId"] = actionId;
resDoc["deviceId"] = deviceId;
resDoc["action"] = action;
resDoc["status"] = statusResult;

char resOut[200];
serializeJson(resDoc, resOut);

// Gửi trạng thái phản hồi
String statusTopic = "iot/devices/" + String(deviceId) + "/status";

client.publish(statusTopic.c_str(), resOut);

Serial.println(
    ">> Đã gửi phản hồi: " + String(resOut) + " vào topic: " + statusTopic);
}

```

Hình 20: Hàm xử lý lệnh điều khiển và phản hồi chéo

- Chức năng: Hàm callback được thư viện PubSubClient gọi khi client.loop() xử lý được một MQTT message mới trên topic đã subscribe.
 - + Giải mã dữ liệu (Parse JSON): Đọc chuỗi tin nhắn nhận được và dùng thư viện ArduinoJson để trích xuất các thông tin: ID tác vụ (actionId), ID thiết bị (deviceId) và Lệnh (action).

- + Định tuyến thiết bị: Dựa vào deviceId để ánh xạ tới đúng chân phần cứng tương ứng (Ví dụ: ID 1 map với chân D5).
- + Thực thi và Kiểm tra chéo (Cross-check): Xuất tín hiệu điện (digitalWrite) để bật/tắt thiết bị. Tiếp theo, hệ thống dùng digitalRead để đọc ngược lại chính chân đó, kiểm tra xem điện áp đã thực sự thay đổi chưa. Nếu khớp, trạng thái được đánh giá là SUCCESS, ngược lại là FAILED.
- + Phản hồi (Publish): Đóng gói ID tác vụ và kết quả (SUCCESS/FAILED) thành một chuỗi JSON mới, sau đó gửi ngược lên topic /status để Backend cập nhật lên Database và giao diện.

5. Vòng lặp chính và cơ chế đọc cảm biến Non-blocking

```
void loop() {
    if (!client.connected()) {
        reconnect();
    }
    client.loop(); // Duy trì kết nối nhận lệnh
    if (millis() - lastSensorTime > 2000) {
        lastSensorTime = millis();
        float h = dht.readHumidity();
        float t = dht.readTemperature();
        int l = 1023 - analogRead(LDR_PIN); // Đọc giá trị ánh sáng (0 - 1023)
        if (isnan(h) || isnan(t)) {
            Serial.println("Lỗi: Không đọc được dữ liệu từ DHT11");
            return;
        }
        // Đóng gói JSON gửi dữ liệu môi trường
        StaticJsonDocument<200> sensorDoc;
        sensorDoc["temperature"] = t;
        sensorDoc["humidity"] = h;
        sensorDoc["light"] = l;
        char sensorOut[200];
        serializeJson(sensorDoc, sensorOut);
        client.publish("iot/sensors/data", sensorOut);
        Serial.println("Đã gửi cảm biến: " + String(sensorOut));
    }
}
```

Hình 21: Hàm thu thập cảm biến

- Chức năng: Là vòng lặp vô tận xử lý đồng thời việc lắng nghe kết nối và định kỳ gửi dữ liệu môi trường.
 - + Duy trì MQTT: Gọi client.loop() để hệ thống liên tục kiểm tra gói tin đến và giữ cho kết nối MQTT không bị ngắt.
 - + Bộ đếm thời gian (Timer): Sử dụng hàm millis() để tạo chu kỳ 2 giây.

- + Đọc và lọc dữ liệu: Đọc nhiệt độ, độ ẩm và cường độ ánh sáng. Nếu cảm biến DHT bị lỗi (trả về giá trị NaN), vòng lặp sẽ bỏ qua việc gửi để tránh rác dữ liệu.
- + Gửi dữ liệu: Đóng gói 3 thông số môi trường thành chuỗi JSON và đẩy lên topic `iot/sensors/data` cho Backend.

III. Luồng điều khiển thiết bị

1. *Nạp mã nguồn từ Arduino vào ESP8266*

- Thiết lập môi trường và kết nối: Vi điều khiển ESP8266 được kết nối với máy tính thông qua cáp Micro-USB (hỗ trợ truyền dữ liệu). Trên phần mềm Arduino IDE, hệ thống được cấu hình nhắm mục tiêu đến board mạch NodeMCU 1.0 (ESP-12E Module) và lựa chọn cổng giao tiếp COM3.
- Biên dịch mã nguồn: Quá trình biên dịch giúp phát hiện các lỗi cú pháp, kiểu dữ liệu và lỗi biên dịch trước khi chương trình được nạp lên vi điều khiển.
- Nạp chương trình: Tập nhị phân được truyền tuần tự qua giao diện Serial UART và ghi trực tiếp vào bộ nhớ Flash của vi điều khiển.
- Kiểm thử và giám sát: Ngay sau khi quá trình nạp đạt 100%, ESP8266 sẽ tự động khởi động lại (Reset). Công cụ Serial Monitor trên Arduino IDE được thiết lập ở tốc độ baud định mức (ví dụ: 115200) để giám sát trực tiếp các bản tin log khởi tạo, qua đó xác minh trạng thái cấp phát IP từ mạng WiFi và tiến trình kết nối thành công với máy chủ MQTT Broker.

2. *Khởi động máy chủ MQTT*

- Lệnh: `.\mosquitto.exe -c mosquitto.conf -v`
 - + `.\mosquitto.exe`: Gọi và chạy file thực thi của phần mềm Mosquitto nằm trong thư mục hiện tại.
 - + `-c mosquitto.conf`: Cờ `-c` (viết tắt của config) yêu cầu Mosquitto khởi động và nạp các cài đặt từ file `mosquitto.conf`.
 - + `-v`: Cờ `-v` (viết tắt của verbose) bật chế độ log chi tiết. Khi có cờ này, mọi hoạt động của broker (như khi có một thiết bị kết nối, ngắt kết nối, gửi tin nhắn publish, hoặc đăng ký subscribe) đều sẽ được in trực tiếp lên màn hình console.

```
Administrator: Windows Powe X + v
PS C:\Program Files\mosquitto> .\mosquitto.exe -c mosquitto.conf -v
1787652165: mosquitto version 2.1.2 starting
1787652165: Config loaded from mosquitto.conf.
1787652165: Bridge support available.
1787652165: Persistence support available.
1787652165: TLS support available.
1787652165: TLS-PSK support available.
1787652165: Websockets support available.
1787652165: Opening ipv4 listen socket on port 1883.
1787652165: mosquitto version 2.1.2 running
```

Hình 22: Giao diện thực thi lệnh khởi động máy chủ MQTT

3. Luồng nhận dữ liệu cảm biến

- Sau khi khởi động máy chủ MQTT, thực hiện kết nối và nạp mã nguồn vào ESP8266, định kỳ 2s ESP8266 sẽ gửi dữ liệu cảm biến gồm 3 giá trị là nhiệt độ, độ ẩm, ánh sáng về máy tính.
- Truy cập topic “iot/sensors/data” để nhận dữ liệu cảm biến.
Lệnh: `.\mosquitto_sub -h localhost -p 1884 -u "NguyenDuyThanh" -P "B23DCCN761" -t "iot/sensors/data" -v`

```
PS C:\Program Files\Mosquitto> .\mosquitto_sub -h localhost -p 1884 -u "NguyenDuyThanh" -P "B23DCCN761" -t "iot/sensors/data" -v
iot/sensors/data {"temperature":27.2,"humidity":54.8,"light":999}
iot/sensors/data {"temperature":27.3,"humidity":57,"light":1023}
iot/sensors/data {"temperature":27.3,"humidity":57,"light":1021}
iot/sensors/data {"temperature":27.3,"humidity":56.9,"light":1003}
iot/sensors/data {"temperature":27.3,"humidity":56.9,"light":1008}
```

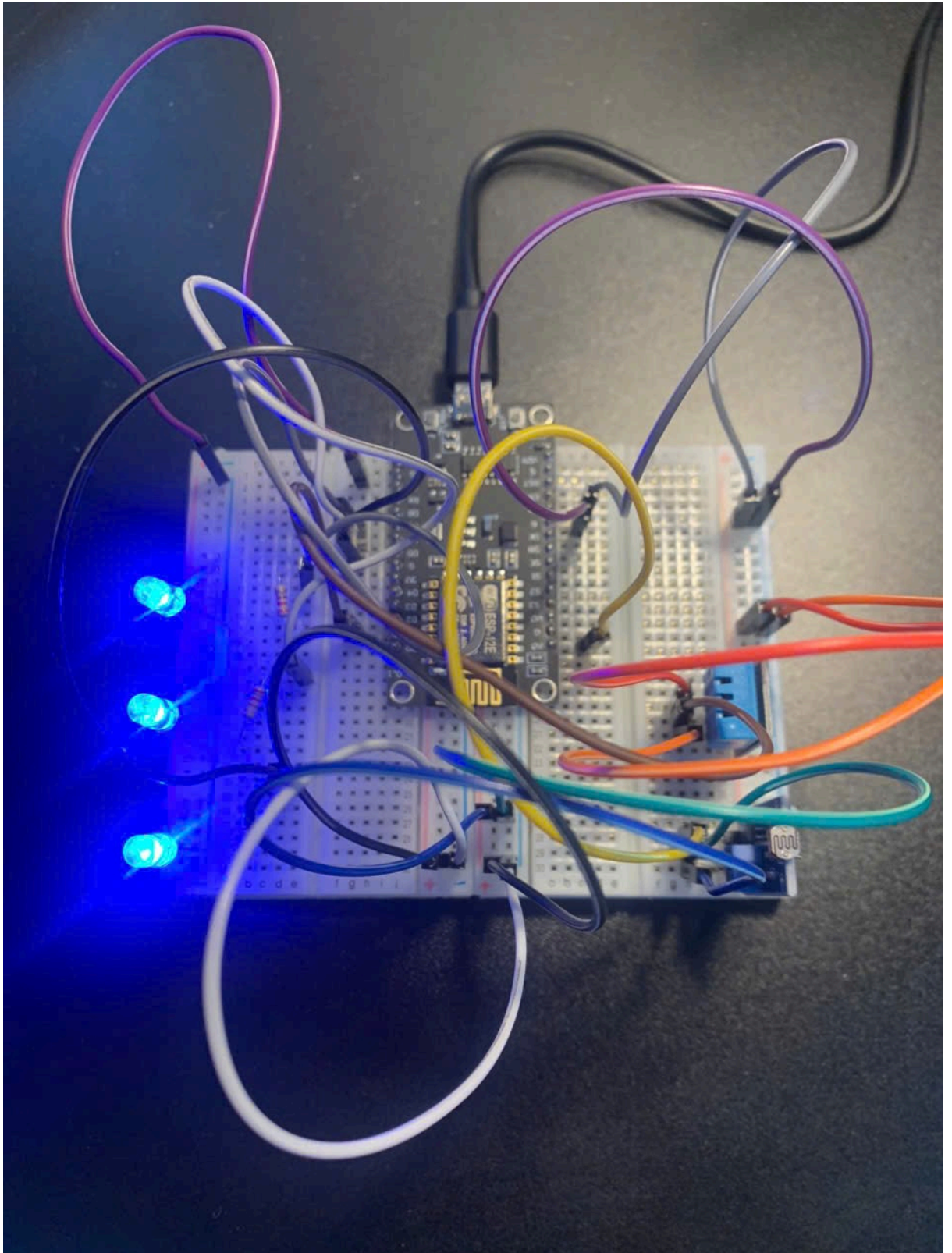
Hình 23: Dữ liệu cảm biến được đẩy lên liên tục mỗi 2 giây

4. Luồng điều khiển đèn LED và nhận kết quả trạng thái

- Lệnh bật cả 3 đèn:
`.\mosquitto_pub -h localhost -p 1884 -u "NguyenDuyThanh" -P "B23DCCN761" -t "iot/devices/0/action" -m '{"actionId": 1, "deviceId": 0, "action": "ON"}'`
- Muốn bật riêng lẻ từng đèn thì chỉ cần thay deviceId tương ứng.
- Lệnh subscribe lắng nghe topic iot/devices/+/status để nhận trạng thái của các đèn sau khi bật/ tắt:
`.\mosquitto_sub -h localhost -p 1884 -u "NguyenDuyThanh" -P "B23DCCN761" -t "iot/devices/+/status" -v`

```
PS C:\Program Files\Mosquitto> .\mosquitto_sub -h localhost -p 1884 -u "NguyenDuyThanh" -P "B23DCCN761" -t "iot/devices/  
+/status" -v  
iot/devices/0/status {"actionId":1,"deviceId":0,"action":"OFF","status":"SUCCESS"}
```

Hình 24: Phản hồi từ ESP8266 sau khi thực hiện tắt cả 3 đèn



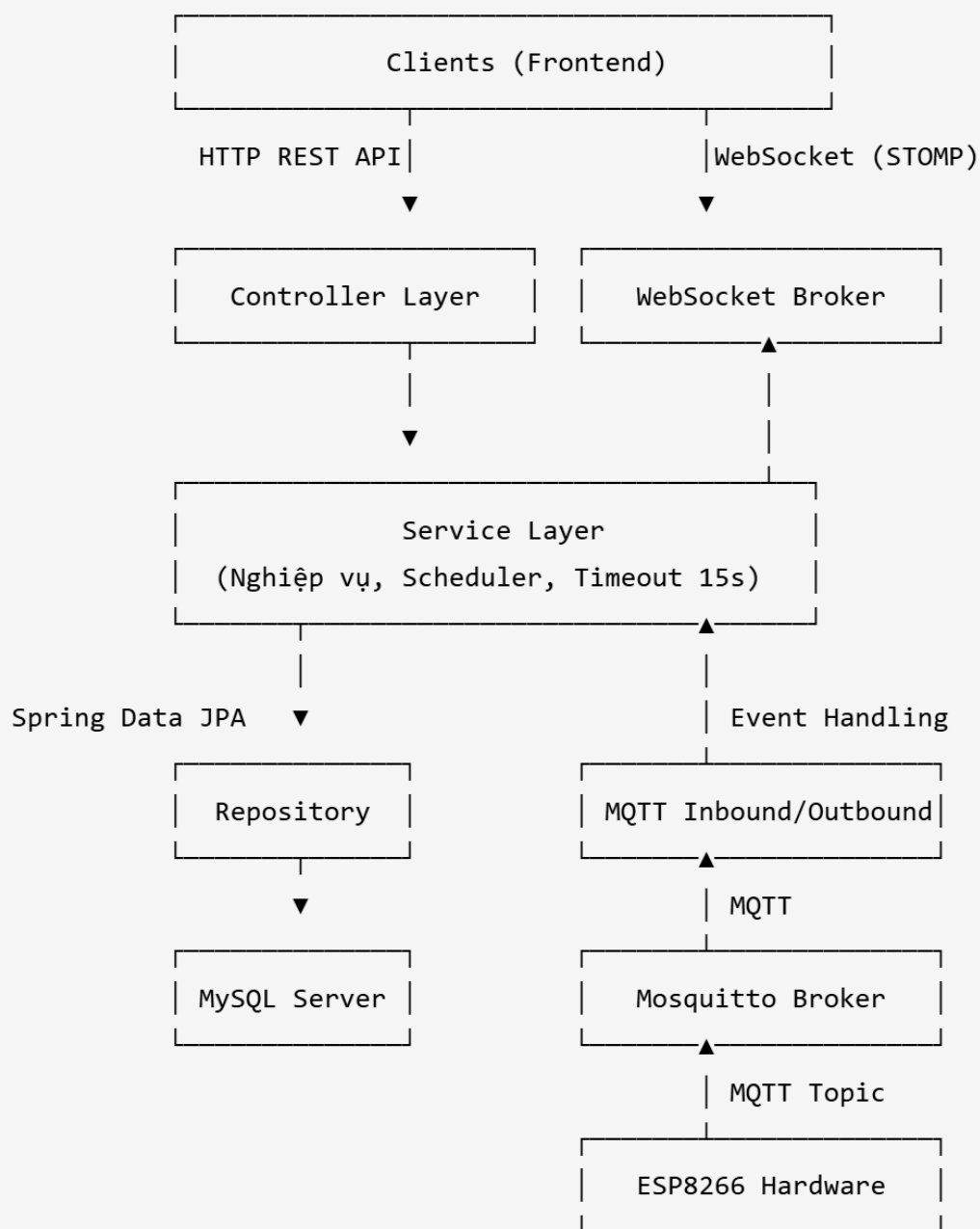
Hình 25: Phần cứng sau khi thực hiện thao tác điều khiển

CHƯƠNG 5: MÃ NGUỒN

I. Back-end

1. Công nghệ và kiến trúc hệ thống

- Hệ thống Back-end được phát triển dựa trên mô hình phân lớp chuẩn doanh nghiệp, phân tách rõ ràng trách nhiệm giữa các tầng nhằm đảm bảo tính độc lập, khả năng mở rộng và dễ bảo trì:
 - + Ngôn ngữ & Nền tảng: Java 21 (LTS), Spring Boot 3.5.0.
 - + Giao tiếp dữ liệu & bền vững: Spring Data JPA / Hibernate kết nối cơ sở dữ liệu quan hệ MySQL.
 - + Tích hợp IoT & Message Broker: Spring Integration MQTT (kết nối Mosquitto Broker qua giao thức MQTT QoS 1).
 - + Truyền thông thời gian thực: Spring WebSocket kết hợp giao thức STOMP và SockJS fallback.
 - + Mô hình phân tầng:
 - + Controller Layer: Tiếp nhận HTTP Request, validate tham số và trả về phản hồi chuẩn hóa ApiResponse<T>.
 - + Service Layer: Triển khai toàn bộ quy tắc nghiệp vụ, điều phối dữ liệu giữa DB, MQTT và WebSocket Broker.
 - + Repository Layer: Kế thừa JpaRepository và JpaSpecificationExecutor để thực hiện truy vấn và lọc dữ liệu động.
 - + MQTT & WebSocket Handlers: Xử lý các luồng bất đồng bộ (Asynchronous Event-driven).



Hình 26: Sơ đồ kiến trúc phân tầng Back-end và các luồng giao tiếp

2. Cấu trúc thư mục back-end

```
iot_backend/src/main/java/edu/ptit/iot/
├─ IotApplication.java          # Điểm khởi chạy ứng dụng Spring Boot
├─ config/                      # Cấu hình hệ thống
│  └─ MqttConfig.java          # Cấu hình MQTT Paho Inbound/Outbound Channel Adapter
│  └─ WebSocketConfig.java     # Cấu hình STOMP Endpoint (/ws/sensors, /ws/devices)
├─ controller/                 # Tầng tiếp nhận REST API
│  └─ ActionController.java    # API lịch sử thao tác bật/tắt thiết bị
│  └─ DataSensorController.java # API dữ liệu cảm biến và biểu đồ
│  └─ DeviceController.java    # API danh sách thiết bị và gửi lệnh điều khiển
├─ dto/                        # Đối tượng truyền tải dữ liệu (Data Transfer Object)
│  └─ request/                 # DTO nhận từ Client: ActionRequest, MqttSensorMessage
│  └─ response/                # DTO chuẩn phản hồi: ApiResponse, PageResponse, v.v.
├─ entity/                    # Tầng thực thể ORM (JPA Entities)
│  └─ Action.java              # Thực thể lịch sử bật/tắt thiết bị
│  └─ DataSensor.java          # Thực thể lưu số đo cảm biến theo thời gian
│  └─ Device.java              # Thực thể thông tin thiết bị (LED)
│  └─ Sensor.java              # Thực thể định nghĩa loại cảm biến
│  └─ User.java                # Thực thể người dùng thao tác
├─ exception/                  # Xử lý lỗi tập trung
│  └─ GlobalExceptionHandler.java # Bắt và chuẩn hóa lỗi trả về cho Client
│  └─ ResourceNotFoundException.java # Ngoại lệ không tìm thấy tài nguyên
├─ mqtt/                      # Bộ xử lý bản tin MQTT
│  └─ MqttMessageHandler.java  # Bóc tách JSON từ Mosquitto, ghi DB và kích hoạt WebSocket
├─ repository/                 # Tầng giao tiếp CSDL (Spring Data JPA)
│  └─ ActionRepository.java     # Truy vấn Action kết hợp JPA Specification
│  └─ DataSensorRepository.java # Truy vấn DataSensor kết hợp phân trang
│  └─ DeviceRepository.java     # Quản lý thiết bị
├─ service/                    # Tầng xử lý nghiệp vụ
│  └─ ActionService.java        # Interface nghiệp vụ thiết bị & hành động
│  └─ DataSensorService.java    # Interface nghiệp vụ dữ liệu cảm biến
│  └─ DeviceService.java        # Interface nghiệp vụ danh sách thiết bị
│  └─ impl/                     # Hiện thực chi tiết nghiệp vụ
│     └─ ActionServiceImpl.java
│     └─ DataSensorServiceImpl.java
│     └─ DeviceServiceImpl.java
```

Hình 27: Cấu trúc thư mục backend

3. Tầng dữ liệu và ánh xạ thực thể (Entity Mapping)

- Cơ sở dữ liệu được ánh xạ bằng Hibernate JPA, đảm bảo ràng buộc toàn vẹn dữ liệu:

Entity	CSDL	Trường dữ liệu	Quan hệ thực thể
Device	devices	id(PK), name, created_at	1-N với Action

Sensor	sensors	id(PK), name, created_at	1-N với Datasensor
User	users	id(PK), username, fullName, password, created_at	1-N với Action
Datasensor	datasensors	id(PK), sensor_id(FK), value, created_at	N-1 với Sensor
Action	actions	id(PK), device_id(FK), user_id(FK), action, status, created_at	N-1 với Device, User

Bảng 7: Cơ sở dữ liệu ánh xạ giữa backend và database

4. Danh mục API và giao thức truyền thông

Method	Endpoint	Chức năng nghiệp vụ	Request Body / Params	Phản hồi chính
GET	/api/datasensors/latest	Lấy giá trị mới nhất của 3 cảm biến	Không	ApiResponse<LatestDataSensorResponse>
GET	/api/datasensors/chart/latest	Lấy 12 mốc gần nhất khởi tạo biểu đồ	Không	ApiResponse<List<LatestDataSensorResponse>>
GET	/api/datasensors	Tra cứu phân trang dữ liệu cảm biến	page, size, type, time, search, searchMode	ApiResponse<PageResponse<DataSensorResponse>>
GET	/api/devices	Lấy danh sách thiết bị & trạng thái mới nhất	Không	ApiResponse<List<DeviceResponse>>
POST	/api/devices/{id}/actions	Gửi lệnh Bật/Tắt thiết bị (ON/OFF)	{ "userId": 1, "action": "ON" }	ApiResponse<ActionResponse> (PENDING)
GET	/api/actions	Tra cứu phân trang lịch sử thao tác	page, size, device, action, status, time	ApiResponse<PageResponse<ActionResponse>>

Bảng 8: Bảng danh mục API

Giao thức	Tên Topic / Channel	Hướng truyền thông	Mục đích
MQTT	iot/sensors/data	ESP8266 → Backend	ESP8266 gửi nhiệt độ, độ ẩm, ánh sáng chu kỳ 2s/lần
MQTT	iot/devices/{deviceId}/action	Backend → ESP8266	Backend truyền lệnh điều khiển (actionId, deviceId, action)
MQTT	iot/devices/{deviceId}/status	ESP8266 → Backend	ESP8266 xác nhận kết quả bật/tắt (SUCCESS / FAILED)
STOMP	/ws/sensors → /topic/sensors	Backend → Frontend	Đẩy dữ liệu cảm biến thời gian thực đến Dashboard
STOMP	/ws/devices → /topic/devices/{id}/status	Backend → Frontend	Đẩy trạng thái điều khiển thiết bị cuối cùng (Success/Timeout)

Bảng 9: Danh mục Topics MQTT và Kênh WebSocket STOMP

5. Cấu hình và xử lý lỗi tập trung

- Cấu hình MQTT Broker
([MqttConfig.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/config/MqttConfig.java)): Kết nối Mosquitto tại tcp://localhost:1884, đăng ký lắng nghe chính xác 2 topic: iot/sensors/data và iot/devices/+/status.
- Cấu hình WebSocket Broker
([WebSocketConfig.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/config/WebSocketConfig.java)): Mở 2 endpoint STOMP độc lập: /ws/sensors và /ws/devices với tiền tố broker /topic.

- Xử lý Ngoại lệ Tập trung
([GlobalExceptionHandler.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/exception/GlobalExceptionHandler.java)): Bắt tất cả lỗi hệ thống (ResourceNotFoundException, ngoại lệ validate, lỗi kết nối) và đóng gói thành cấu trúc đồng nhất:

```
{
  "success": false,
  "message": "Chi tiết thông điệp lỗi",
  "data": null
}
```

II. Front-end

1. Công nghệ và kiến trúc phía Client

Giao diện ứng dụng được xây dựng theo kiến trúc Single Page Application (SPA) hiện đại:

- Nền tảng: React 19, TypeScript, công cụ đóng gói Vite.
- Giao diện & Styling: Tailwind CSS 4, Lucide Icons, React Icons.
- Trực quan hóa dữ liệu: Thư viện Recharts (LineChart, AreaChart với gradient động).
- Truyền thông API & Socket: Axios (REST API Client) kết hợp @stomp/stompjs và sockjs-client cho kênh kết nối Realtime hai chiều.

2. Cấu trúc thư mục Front-end

```

iot_frontend/src/
├─ App.tsx                # Định tuyến ứng dụng React Router DOM
├─ main.tsx               # Điểm nạp React DOM
├─ api/                   # Tầng kết nối REST API tập trung
│  └─ axiosClient.ts      # Khởi tạo Axios instance, cấu hình BaseURL & Interceptors
│  └─ types.ts            # Định nghĩa toàn bộ Interface/Type TypeScript
│  └─ sensorApi.ts        # Các hàm gọi API cảm biến và biểu đồ
│  └─ deviceApi.ts        # Các hàm gọi API thiết bị và gửi lệnh điều khiển
│  └─ actionApi.ts        # Các hàm gọi API tra cứu lịch sử thao tác
├─ services/              # Dịch vụ truyền thông nền
│  └─ websocketService.ts  # Quản lý 2 kết nối STOMP Clients độc lập (/ws/sensors, /ws/devices)
├─ layouts/               # Khung giao diện dùng chung
│  └─ MainLayout.tsx      # Sidebar thu phóng, Header và Avatar cá nhân
└─ pages/                 # Các trang chức năng chính
   └─ Dashboard.tsx       # Trang giám sát cảm biến realtime & điều khiển 3 đèn LED
   └─ DataSensor.tsx      # Trang bảng dữ liệu cảm biến có lọc & phân trang server
   └─ History.tsx         # Trang lịch sử thao tác thiết bị có lọc & phân trang server
   └─ Profile.tsx         # Trang hồ sơ cá nhân và liên kết đồ án

```

Hình 28: Cấu trúc thư mục Front-end

3. Điều hướng Ứng dụng (Routing)

Đường dẫn (URL)	Component hiển thị	Chức năng nghiệp vụ
/	Navigate	Tự động chuyển hướng về /dashboard
/dashboard	Dashboard.tsx	Xem giá trị cảm biến tức thời, biểu đồ 12 điểm và bật/tắt 3 đèn LED
/datasensor	DataSensor.tsx	Tra cứu bảng dữ liệu cảm biến phân trang, lọc theo loại và mức đo
/history	History.tsx	Tra cứu lịch sử điều khiển bật/tắt thiết bị theo trạng thái và thời gian
/profile	Profile.tsx	Hiển thị thông tin sinh viên, đổi ảnh đại diện và liên kết báo cáo

Bảng 10: Danh mục các đường dẫn (Routes) trên Front-end

4. Tầng giao tiếp dữ liệu: REST API Client & WebSocket Service

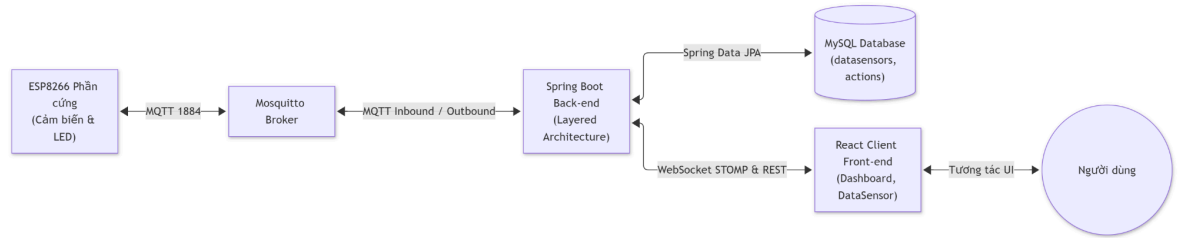
- axios Client tập trung
([axiosClient.ts](file:///d:/privateProject/iot_project/iot/iot_frontend/src/api/axiosClient.ts)): Cấu hình baseUrl: http://localhost:8080/api, timeout 10 giây và gắn Interceptor tự động bóc tách response.data nhằm giải phóng mã nguồn các trang khỏi việc parse JSON lặp lại.
- Dịch vụ WebSocket 2 kênh độc lập
([websocketService.ts](file:///d:/privateProject/iot_project/iot/iot_frontend/src/services/websocketService.ts)): Quản lý riêng 2 thực thể Client của STOMP:
 - + sensorClient: Kết nối tới http://localhost:8080/ws/sensors, chuyên lắng nghe /topic/sensors.
 - + deviceClient: Kết nối tới http://localhost:8080/ws/devices, chuyên lắng nghe /topic/devices/{id}/status.

5. Quản lý trạng thái (State Management) & đồng bộ bộ nhớ

- Hệ thống kết hợp 3 tầng quản lý trạng thái:
 - + React State (useState): Quản lý trạng thái cục bộ của giao diện (danh sách dữ liệu, cờ loading, dữ liệu biểu đồ).
 - + URL Search Parameters (useSearchParams): Đồng bộ trực tiếp các bộ lọc (page, size, type, time, search) lên thanh địa chỉ trình duyệt, giúp người dùng có thể lưu bookmark hoặc chia sẻ liên kết có sẵn bộ lọc.
 - + Storage (sessionStorage & localStorage): Lưu trữ tạm thời trạng thái lọc và công tắc đèn, ngăn ngừa việc mất dữ liệu khi nhấn F5 tải lại trang trước khi dữ liệu từ máy chủ được đồng bộ lại.

III. Các luồng nghiệp vụ cốt lõi

1. Kiến trúc tương tác tổng quát



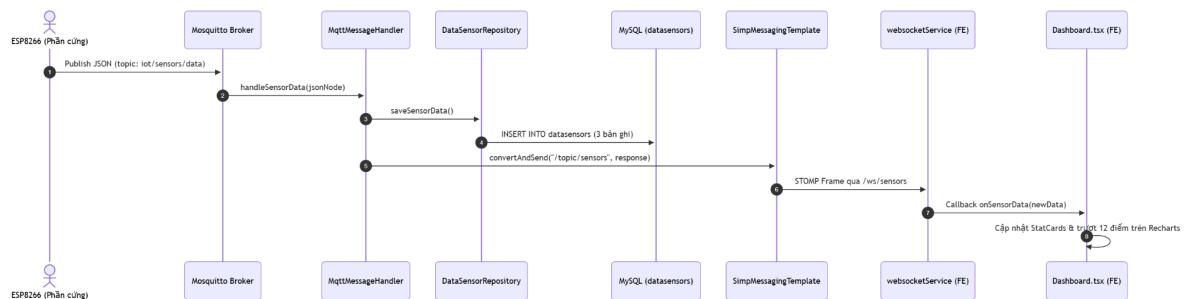
Hình 29: Sơ đồ tương tác tổng quan

STT	Luồng nghiệp vụ	Tham chiếu	Giao thức / API / Topic	Lớp & Tập xử lý chính	Bảng CSDL
1	Thu thập & Giám sát Cảm biến Realtime	UC-01	Topic: <code>iot/sensors/data</code> STOMP: <code>/topic/sensors</code>	MqttMessageHandler DataSensorRepository websocketService, Dashboard	datasensors
2	Điều khiển Thiết bị & Timeout 15s	UC-02	REST: POST <code>/api/devices/{id}/actions</code> Topic: <code>iot/devices/{id}/action</code> STOMP: <code>/topic/devices/{id}/status</code>	ApiController ActionServiceImpl ActionRepository deviceApi, Dashboard	devices actions
3	Tra cứu & Phân trang Lịch sử	UC-03	REST: GET <code>/api/datasensors</code>	DataSensorController DataSensorServiceImpl DataSensorRepository sensorApi, DataSensor	datasensors

Bảng 11: Tổng hợp các luồng nghiệp vụ cốt lõi của hệ thống

2. Luồng 1: Thu thập và giám sát cảm biến thời gian thực

- Mô tả: Vi điều khiển ESP8266 tự động kích hoạt định kỳ 2 giây/lần nhằm gửi dữ liệu nhiệt độ, độ ẩm và ánh sáng lên hệ thống. Dữ liệu được bóc tách để lưu vĩnh viễn vào cơ sở dữ liệu và đồng thời phát tán trực tiếp lên bảng điều khiển Web qua WebSocket STOMP mà không cần tải lại trang (Tham chiếu: UC-01).
- Sơ đồ tuần tự qua các lớp:

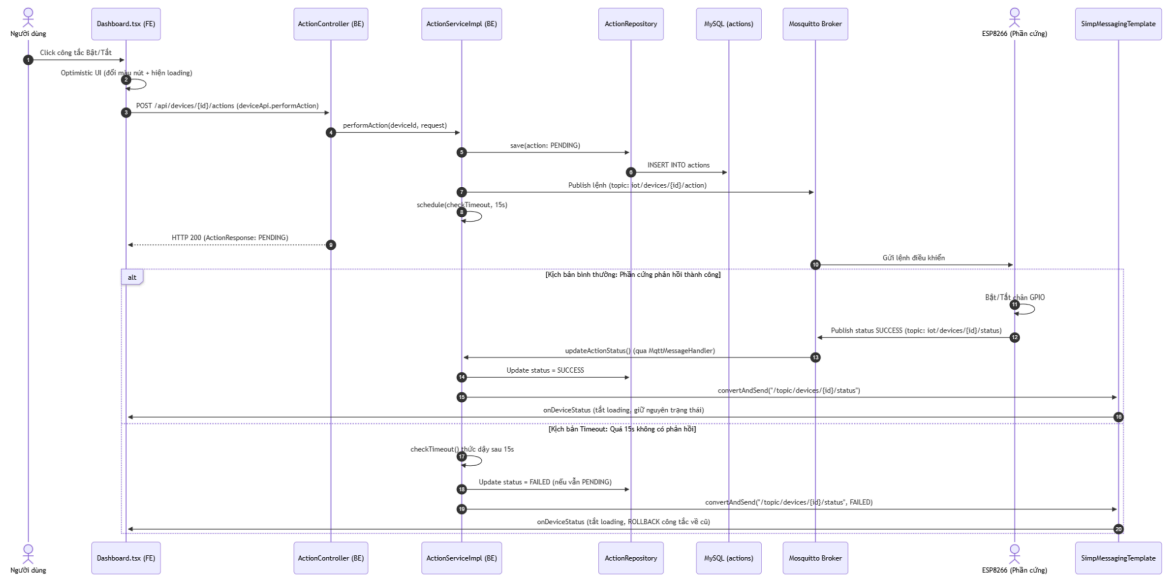


Hình 30: Luồng thu thập và giám sát cảm biến thời gian thực

- Các bước thực hiện:

- ESP8266 Firmware: Đọc chỉ số DHT11 và LDR, gửi JSON qua MQTT topic `iot/sensors/data`.

2. `MqttMessageHandler.handleSensorData()`: Nhận gói tin từ Broker, bóc tách chuỗi JSON 3 cảm biến
(`[MqttMessageHandler.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/mqtt/MqttMessageHandler.java)`)).
 3. `DataSensorRepository.save()`: Lưu 3 bản ghi riêng biệt (temperature, humidity, light) vào bảng `datasensors`
(`[DataSensorRepository.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/repository/DataSensorRepository.java)`)).
 4. `SimpMessagingTemplate.convertAndSend()`: Gửi dữ liệu tổng hợp qua WebSocket STOMP tới kênh `/topic/sensors`.
 5. `websocketService.onSensorData()`: Nhận frame STOMP và kích hoạt callback dữ liệu tại Front-end
(`[websocketService.ts](file:///d:/privateProject/iot_project/iot/iot_frontend/src/services/websocketService.ts)`)).
 6. `Dashboard.tsx`: Cập nhật trực tiếp số liệu vào 3 thẻ `StatCard` và chuẩn hóa tỉ lệ trượt 12 điểm trên biểu đồ `Recharts`
(`[Dashboard.tsx](file:///d:/privateProject/iot_project/iot/iot_frontend/src/pages/Dashboard.tsx)`)).
- Ngoại lệ chính: Nếu gói tin JSON từ cảm biến bị sai định dạng, `MqttMessageHandler` bắt lỗi, ghi log và bỏ qua, không làm gián đoạn luồng vận hành.
3. ***Luồng 2: Điều khiển thiết bị kết hợp hẹn giờ timeout 15s và cơ chế Rollback***
 - Mô tả: Người dùng nhấn công tắc trên giao diện Web nhằm điều khiển bật/tắt thiết bị ngoại vi (LED/Relay). Hệ thống thực thi bất đồng bộ qua MQTT và kích hoạt bộ hẹn giờ 15 giây để tự động phục hồi (Rollback) trạng thái nếu thiết bị mất kết nối hoặc không phản hồi (Tham chiếu: UC-02).
 - Sơ đồ tuần tự qua các lớp:



Hình 31: Luồng điều khiển thiết bị kết hợp hẹn giờ timeout 15s và cơ chế Rollback

- Các bước thực hiện:

1. Dashboard.handleToggleDevice(): Kích hoạt Optimistic UI đổi màu nút, bật spinner xoay loading ([Dashboard.tsx](file:///d:/privateProject/iot_project/iot/iot_frontend/src/pages/Dashboard.tsx)).
2. deviceApi.performAction(): Gửi HTTP POST /api/devices/{id}/actions lên Back-end ([deviceApi.ts](file:///d:/privateProject/iot_project/iot/iot_frontend/src/api/deviceApi.ts)).
3. ActionController.performAction(): Tiếp nhận request từ Client, chuyển tiếp sang tầng Service ([ActionController.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/controller/ActionController.java)).
4. ActionServiceImpl.performAction(): Lưu bản ghi PENDING vào bảng actions qua ActionRepository, gửi lệnh tới MQTT topic `iot/devices/{id}/action`, và đăng ký tác vụ `scheduler.schedule(checkTimeout, 15, TimeUnit.SECONDS)` ([ActionServiceImpl.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/service/impl/ActionServiceImpl.java)).
5. ESP8266 Firmware: Nhận lệnh từ Broker, đảo chân GPIO và publish trạng thái SUCCESS về topic `iot/devices/{id}/status`.
6. MqttMessageHandler.handleDeviceStatus(): Bắt bản tin phản hồi từ phần cứng, chuyển sang Service

- ([MqttMessageHandler.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/mqtt/MqttMessageHandler.java)).
7. ActionServiceImpl.updateActionStatus(): Kiểm tra nếu trạng thái đang là PENDING, cập nhật thành SUCCESS trong CSDL và bắn tin qua SimpMessagingTemplate tới /topic/devices/{id}/status.
 8. websocketService.onDeviceStatus(): Nhận frame STOMP, Dashboard.tsx tắt loading và giữ nguyên vị trí công tắc ([websocketService.ts](file:///d:/privateProject/iot_project/iot/iot_frontend/src/services/websocketService.ts)).
- Ngoại lệ chính (Xử lý Timeout 15s & Rollback): Nếu sau 15 giây phản ứng không phản hồi, hàm checkTimeout() tự động đánh dấu FAILED và gửi tin qua WebSocket, Front-end lập tức Rollback công tắc về trạng thái cũ và báo lỗi.
 - Trích đoạn mã nguồn cốt lõi xử lý Timeout ([ActionServiceImpl.java:130-145](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/service/impl/ActionServiceImpl.java#L130-L145)):

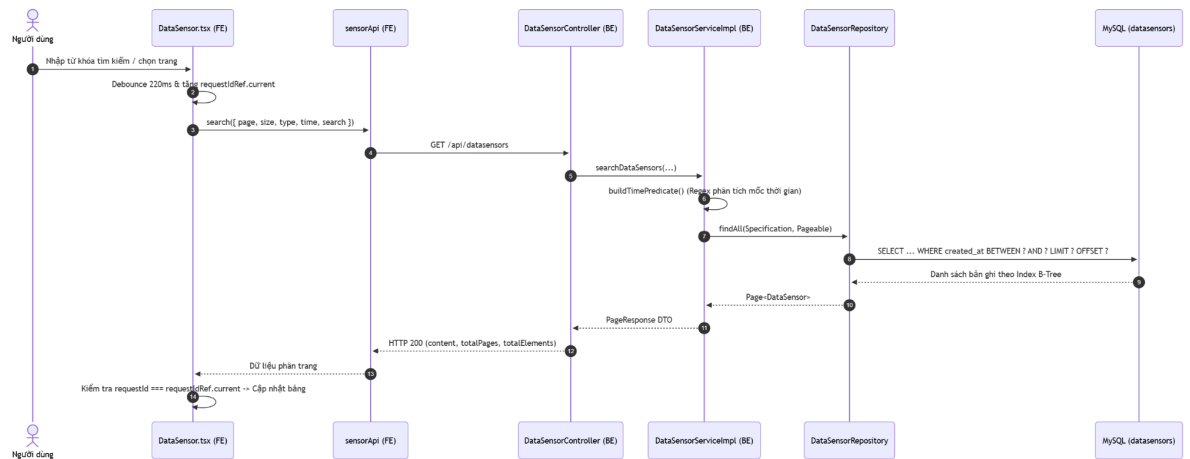
```
// Hẹn giờ kiểm tra sau 15 giây ngay khi tạo lệnh
scheduler.schedule(() -> checkTimeout(action.getId(), deviceId), 15, TimeUnit.SECONDS);

private void checkTimeout(Integer actionId, Integer deviceId) {
    actionRepository.findById(actionId).ifPresent(action -> {
        if ("PENDING".equals(action.getStatus())) { // Chống Race Condition
            action.setStatus("FAILED");
            actionRepository.save(action);
            messagingTemplate.convertAndSend("/topic/devices/" + deviceId + "/status", mapToActionResponse(action));
        }
    });
}
```

Hình 32: Đoạn mã nguồn tiêu biểu

4. Luồng 3: Tra cứu và phân trang lịch sử cảm biến

- Mô tả: Người dùng nhập từ khóa tìm kiếm (loại cảm biến, mốc thời gian) và thực hiện phân trang dữ liệu lịch sử trên giao diện Web. Hệ thống phân tích cú pháp chuỗi bằng Regex và truy vấn phân trang tối ưu bằng chỉ mục B-Tree trong cơ sở dữ liệu để trả kết quả nhanh chóng, mượt mà (Tham chiếu: UC-03).
- Sơ đồ tuần tự qua các lớp:



Hình 33: Luồng tra cứu và phân trang lịch sử cảm biến

- Các bước thực hiện:

1. DataSensor.tsx: Kích hoạt Debounce 220ms khi người dùng nhập từ khóa, tăng biến tham chiếu requestIdRef.current và gọi API tìm kiếm ([DataSensor.tsx](file:///d:/privateProject/iot_project/iot/iot_frontend/src/pages/DataSensor.tsx)).
2. sensorApi.search(): Gửi HTTP GET /api/datasensors kèm các tham số lọc và phân trang lên máy chủ ([sensorApi.ts](file:///d:/privateProject/iot_project/iot/iot_frontend/src/api/sensorApi.ts)).
3. DataSensorController.searchDataSensors(): Tiếp nhận request, chuyển các tham số lọc sang tầng Service ([DataSensorController.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/controller/DataSensorController.java)).
4. DataSensorServiceImpl.buildTimePredicate(): Dùng Regex phân tích chuỗi thời gian (năm, ngày/tháng, giờ:phút) để sinh biểu thức khoảng CriteriaBuilder.between() ([DataSensorServiceImpl.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/service/impl/DataSensorServiceImpl.java)).
5. DataSensorRepository.findAll(): Thực thi truy vấn phân trang thông qua Spring Data JPA Specification, quét trên chỉ mục B-Tree Index của cột created_at trong MySQL ([DataSensorRepository.java](file:///d:/privateProject/iot_project/iot/iot_backend/src/main/java/edu/ptit/iot/repository/DataSensorRepository.java)).

6. DataSensor.tsx: Xác thực requestId === requestIdRef.current (chống Race Condition do mạng lag) trước khi nạp dữ liệu vào bảng và cập nhật số phân trang.
- Ngoại lệ chính: Nếu mạng chậm chờn khiến request cũ phản hồi sau request mới, DataSensor.tsx tự động hủy bỏ kết quả cũ dựa trên requestIdRef, ngăn chặn hoàn toàn việc hiển thị sai lệch dữ liệu.

IV. Cài đặt và triển khai hệ thống

1. Yêu cầu môi trường

- Hệ điều hành: Windows 10/11, Linux (Ubuntu 20.04+) hoặc macOS.
- Java: JDK 21 trở lên (khuyến nghị Eclipse Adoptium Temurin 21).
- Node.js & npm: Node.js v18.0.0+ và npm v9.0.0+.
- Cơ sở dữ liệu: MySQL Server 8.0+ (Cổng mặc định 3306, Database: iot, Username: root).
- MQTT Broker: Eclipse Mosquitto (Chạy tại cổng 1884).

2. Khởi chạy phía Back-end

1. Di chuyển vào thư mục backend

```
cd iot/iot_backend
```

2. Biên dịch và kiểm tra dependencies

```
./mvnw clean compile
```

3. Khởi động ứng dụng Spring Boot

```
./mvnw spring-boot:run
```

Máy chủ Back-end sẽ lắng nghe tại: <http://localhost:8080>.

3. Khởi chạy phía Front-end

1. Di chuyển vào thư mục frontend

```
cd iot/iot_frontend
```

2. Cài đặt các gói phụ thuộc thư viện

```
npm install
```

3. Khởi chạy máy chủ phát triển Vite

```
npm run dev
```

Truy cập bảng điều khiển hệ thống tại: <http://localhost:5173>.

CHƯƠNG 6: KẾT LUẬN

I. Tổng kết các kết quả đạt được

- Xây dựng thành công toàn diện hệ thống IoT từ phần cứng đến phần mềm, bao gồm vi điều khiển ESP8266, cảm biến (DHT11, ánh sáng) và cơ cấu chấp hành (3 đèn LED).

- Triển khai thành công kiến trúc truyền thông qua MQTT Broker (Mosquitto) để truyền nhận dữ liệu và điều khiển thiết bị theo thời gian thực.
- Hoàn thiện hệ thống Backend bằng Spring Boot kết hợp cơ sở dữ liệu MySQL để lưu trữ thông tin, lịch sử cảm biến và lịch sử thao tác.
- Phát triển giao diện Web bằng ReactJS trực quan, hiển thị giá trị thời gian thực, biểu đồ biến thiên và hỗ trợ tra cứu, phân trang lịch sử.
- Ưu điểm của hệ thống:
 - + Đảm bảo tính đồng bộ và thời gian thực nhờ kết hợp giữa MQTT và WebSocket.
 - + Có cơ chế kiểm tra chéo (cross-check) trạng thái phần cứng và xử lý timeout (15 giây) giúp tăng tính ổn định, an toàn.
 - + Giao diện quản lý thân thiện, hỗ trợ lọc và tìm kiếm dữ liệu linh hoạt.
- Hạn chế của hệ thống:
 - + Hệ thống hiện tại mới chỉ tích hợp số lượng cảm biến và thiết bị ở mức cơ bản (nhiệt độ, độ ẩm, ánh sáng và 3 đèn LED) trên phạm vi cục bộ.

II. Hướng phát triển

- Mở rộng thêm các loại cảm biến khác (như cảm biến khí gas, chuyển động PIR, độ ẩm đất).
- Tích hợp tính năng cảnh báo thông minh tự động (gửi thông báo qua Telegram hoặc Email khi thông số môi trường vượt ngưỡng cho phép).
- Nâng cấp hệ thống để triển khai trên Cloud Server thay vì môi trường chạy thử nghiệm cục bộ (localhost).